

词典

散列：循对象访问

We are shaped by our thoughts; we become what we think.

- Buddha

Man's thought is shaped by his tongue.

- Anonymous

于是在熟人中，我们话也少了，我们“眉目传情”，我们“指石相证”，
我们抛开了比较间接的象征原料，而求更直接的会意了

邓俊辉

deng@tsinghua.edu.cn

联合数组：更直接、更有效的访问

❖ 数组？再常见不过，比如：

fib[0] = 0

fib[1] = 1

fib[2] = 1

fib[3] = 2

fib[4] = 3

fib[5] = 5

fib[6] = 8

...

❖ Associative Array

——与常规的数组有何区别？

❖ 根据数据元素的取值，直接访问！

style["关羽"] = "云长"

style["张飞"] = "翼德"

style["赵云"] = "子龙"

style["马超"] = "孟起"

❖ 下标不再是整数，甚至没有大小次序

——更为直观、便捷

❖ 支持的语言：

Snobol4、MUMPS、SETL、Rexx、AWK、

Java、Python、Perl、Ruby、PHP、...

Java: HashMap + Hashtable

```
import java.util.*;

public class Hash {
    public static void main(String[] args) {

        HashMap HM = new HashMap(); //Map
        HM.put("东岳", "泰山"); HM.put("西岳", "华山"); HM.put("南岳", "衡山");
        HM.put("北岳", "恒山"); HM.put("中岳", "嵩山"); System.out.println(HM);

        Hashtable HT = new Hashtable(); //Dictionary
        HT.put("东岳", "泰山"); HT.put("西岳", "华山"); HT.put("南岳", "衡山");
        HT.put("北岳", "恒山"); HT.put("中岳", "嵩山"); System.out.println(HT);

    }
}
```

Perl: %Hash Type

❖ 由字符串 (string) 标识的一组无序标量 (scalar) //亦即MAP

❖ `my %hero = ( "云长"=>"关羽", "翼德"=>"张飞", "子龙"=>"赵云", "孟起"=>"马超") ;`

`foreach $style (keys %hero) # Hash类型的变量由%引导`

`{ print "$style => $hero{$style}\n"; }`

❖ `$hero{"汉升"} = "黄忠";`

`foreach $style (keys %hero)`

`{ print "$style => $hero{$style}\n"; }`

`foreach $style (reverse sort keys %hero)`

`{ print "$style => $hero{$style}\n"; }`

Python: Dictionary Class

❖ `beauty = dict( { "沉鱼":"西施", "落雁":"昭君", "闭月":"貂蝉", "羞花":"玉环" } )`

`print( beauty )`

❖ `beauty["红颜"] = "圆圆"`

`print( beauty )`

❖ `for (alias, name) in beauty.items() :`

`print( alias, ":", name )`

❖ `for alias, name in sorted( beauty.items() ) :`

`print( alias, ":", name )`

❖ `for alias in sorted( beauty.keys(), reverse = True ) :`

`print( alias, ":", beauty[alias] )`

Ruby: Hash Table

```
scarborough = # declare and initialize a hashtable  
  { "P"=>"parsley", "S"=>"sage", "R"=>"rosemary", "T"=>"thyme" }
```

```
puts scarborough # output the hash table
```

```
for k in scarborough.keys # output hash table items
```

```
  puts k + "=>" + scarborough[k] # 1-by-1
```

```
end
```

```
for k in scarborough.keys.sort # output hash table items
```

```
  puts k + "=>" + scarborough[k] # 1-by-1 in order
```

```
end
```

词条 ~ 映射/词典

❖ entry = (key, value)

❖ Map/Dictionary: 词条的集合

- 关键码禁止/允许相等

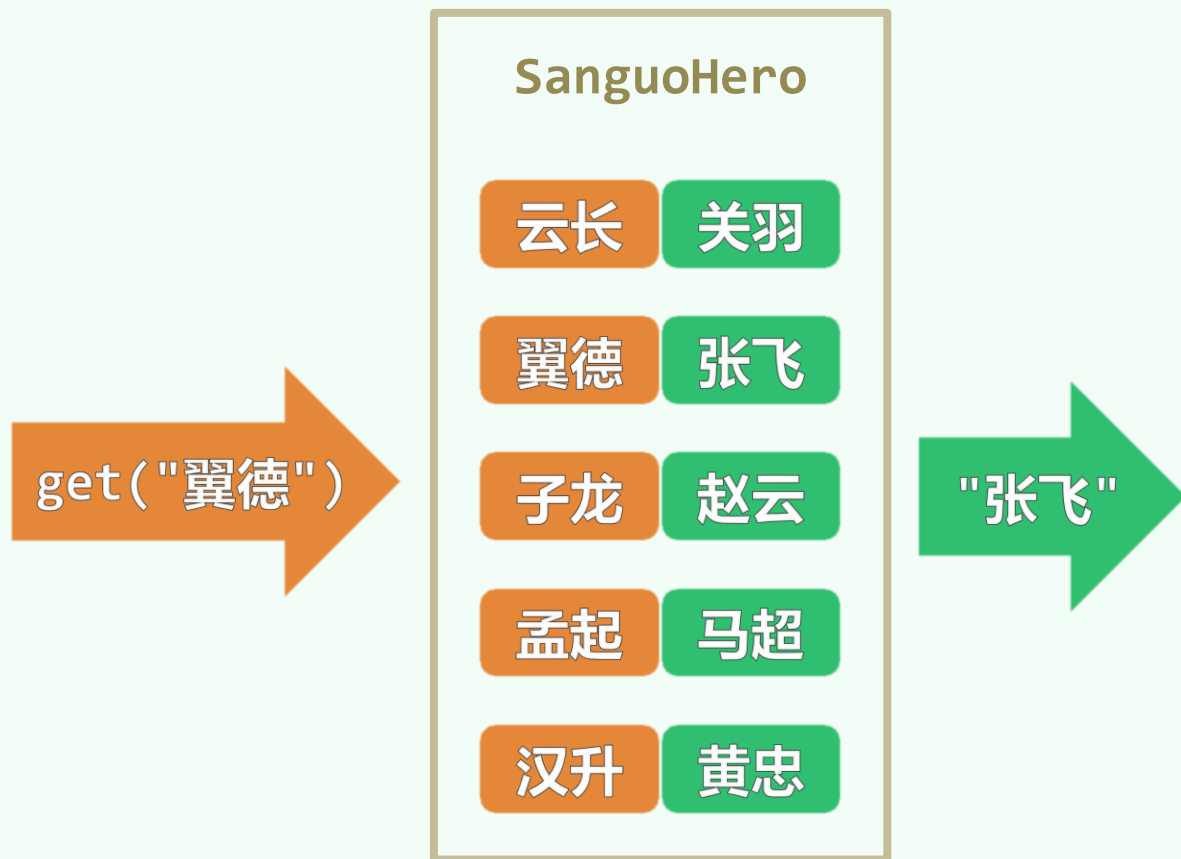
- get(key)

put(key, value)

remove(key)

❖ 关键码未必定义了大小次序

较之BST, 可以支持更多样的元素类型



Dictionary ADT

❖ `template <typename K, typename V> //key、value`

```
struct Dictionary {
```

```
    virtual Rank size() = 0;
```

```
    virtual Entry<K, V>* put( K, V ) = 0;
```

```
    virtual Entry<K, V>* get( K ) = 0;
```

```
    virtual bool remove( K ) = 0;
```

```
};
```

❖ 词典中的词条只需支持**判等/比对**操作，尽管

诸如 `Java::TreeMap` 等实现仍支持**大小/比较器**

`get("翼德")`

SanguoHero	
云长	关羽
翼德	张飞
子龙	赵云
孟起	马超
汉升	黄忠

"张飞"

词典

散列：原理

09-A2

書者，散也。欲書先散懷抱，任情恣性，然後書之

如果是活着的人，想谁，找谁一趟不就完了？

找不得，找不得，当年就是因为个找，我差点丢了命

邓俊辉

deng@tsinghua.edu.cn

电话: 号码 ~ 人

General inquiries

Tel: Toll Free: 1-800-IBM-4YOU

E-mail: askibm@vnet.ibm.com

www.ibm.com/us/en/

Shopping

Tel: Toll Free: 1-888-SHOP-IBM

Sales Center

1-855-2-LENOVO (1-855-253-6686)

Mon - Fri: 9am-9pm (EST)

Sat - Sun: 9am-6pm (EST)

Customer Service

1-855-2-LENOVO (1-855-253-6686)

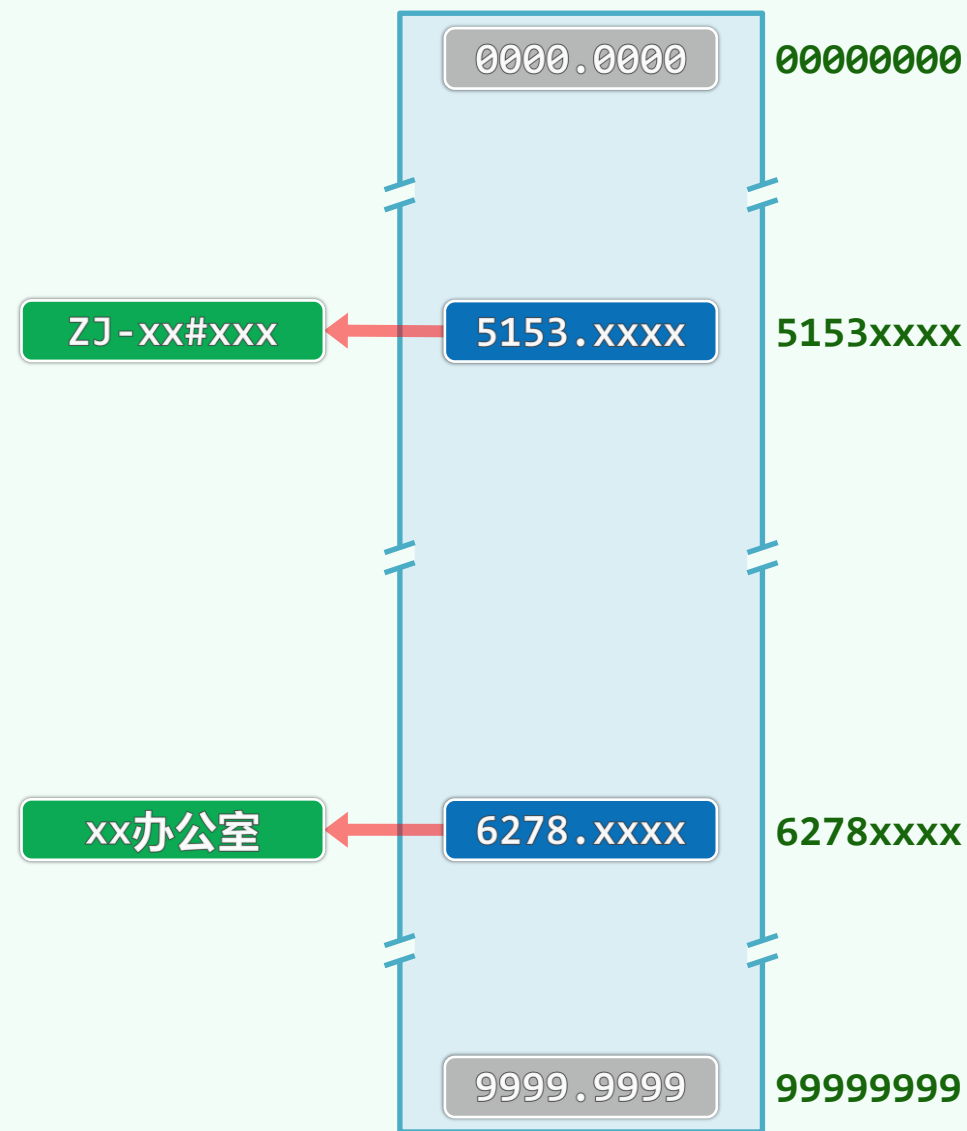
Mon - Fri: 9am-9pm (EST)

Sat - Sun: 9am-6pm (EST)



电话簿

- ❖ 需求：为一所学校制作电话簿
号码 --> 教员、学生、员工、办公室
- ❖ 蛮力：用户记录 ~ 数组Rank ~ 电话号码
 $O(1)$ 效率！
- ❖ 以清华为例（2003）
#可能的电话 = $\mathcal{R} = 10^8 = 100\text{M}$
#实有的电话 = $\mathcal{N} = 25,000 = 25\text{K}$
- ❖ 问题：空间 = $O(\mathcal{R} + \mathcal{N}) = O(100\text{M} + 25\text{K})$
效率 = $25\text{K} / 100\text{M} = 0.025\%$
- ❖ 如何在保持查找速度的同时，降低存储消耗？



散列表 / 散列函数

❖ 桶 (bucket) : 直接存放或间接指向一个词条

❖ Bucket array ~ Hashtable

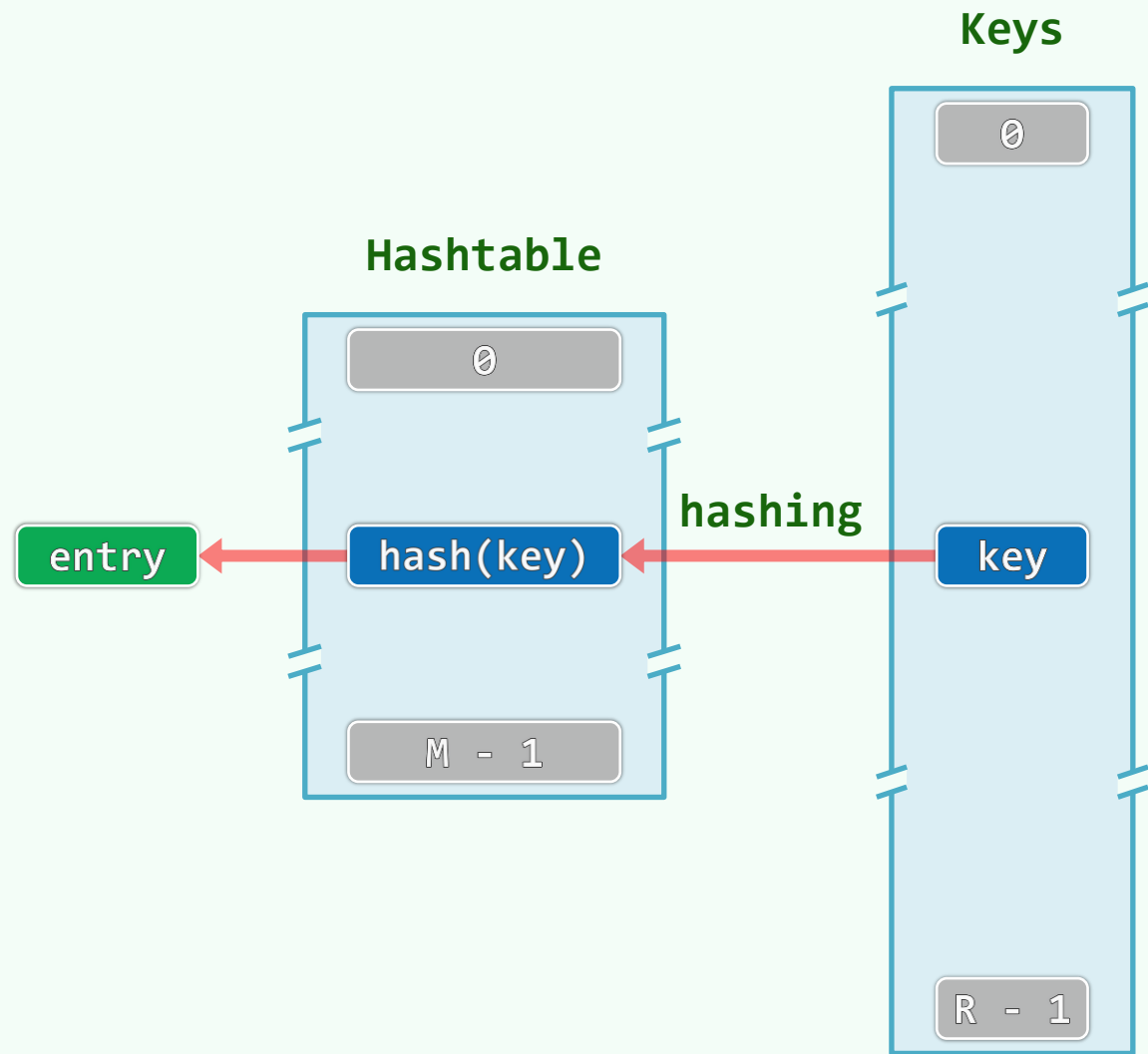
- 容量: M
- 满足: $N < M \ll R$
- 空间: $O(N + M) = O(N)$

❖ 定址/杂凑/散列

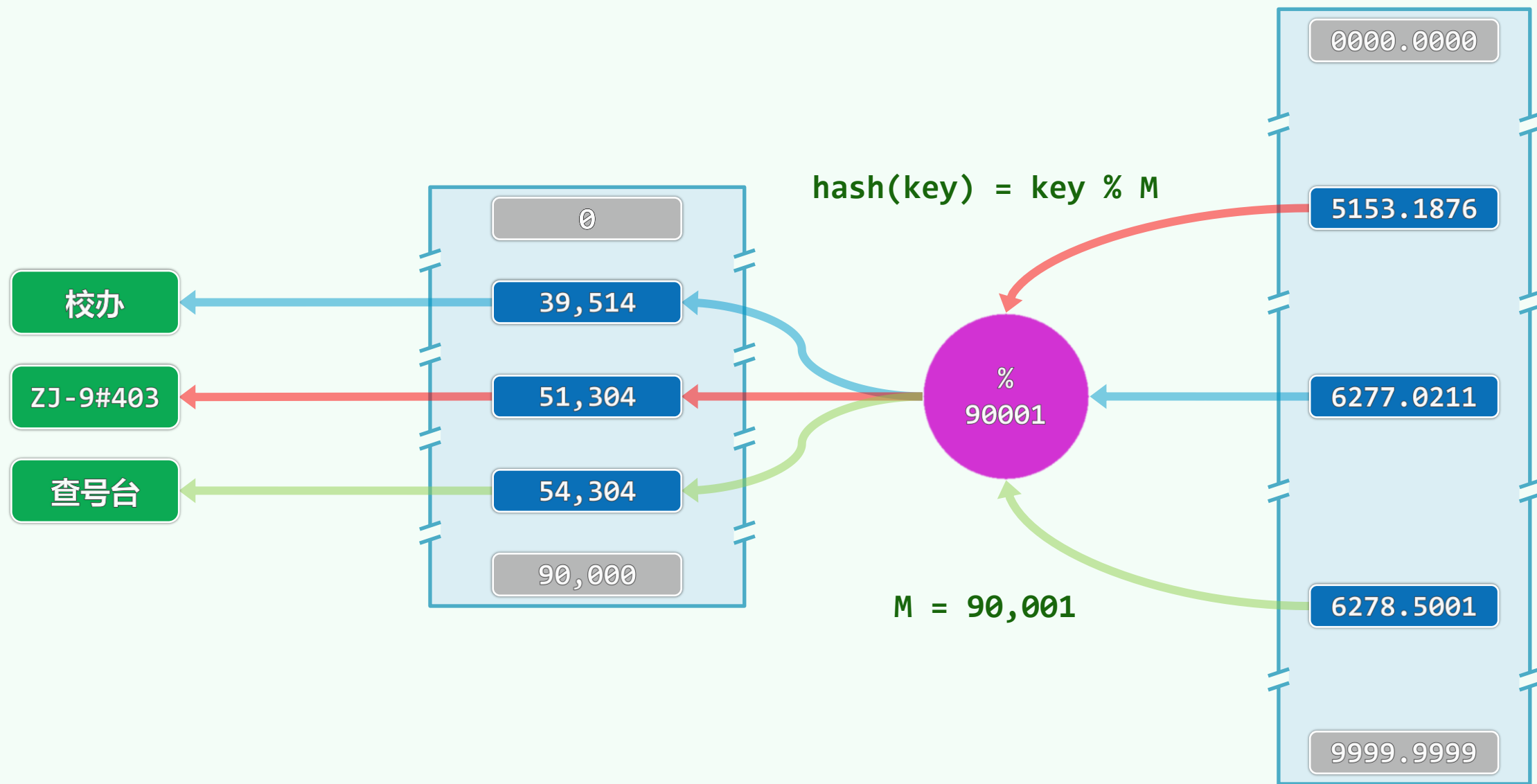
- 根据词条的key (未必可比较)
- “直接” 确定散列表入口 (无论表有多长)

❖ 散列函数: $hash() : key \mapsto \&entry$

❖ “直接” : $expected-O(1) \neq O(1)$



实例



09-A3

词典

散列：冲突

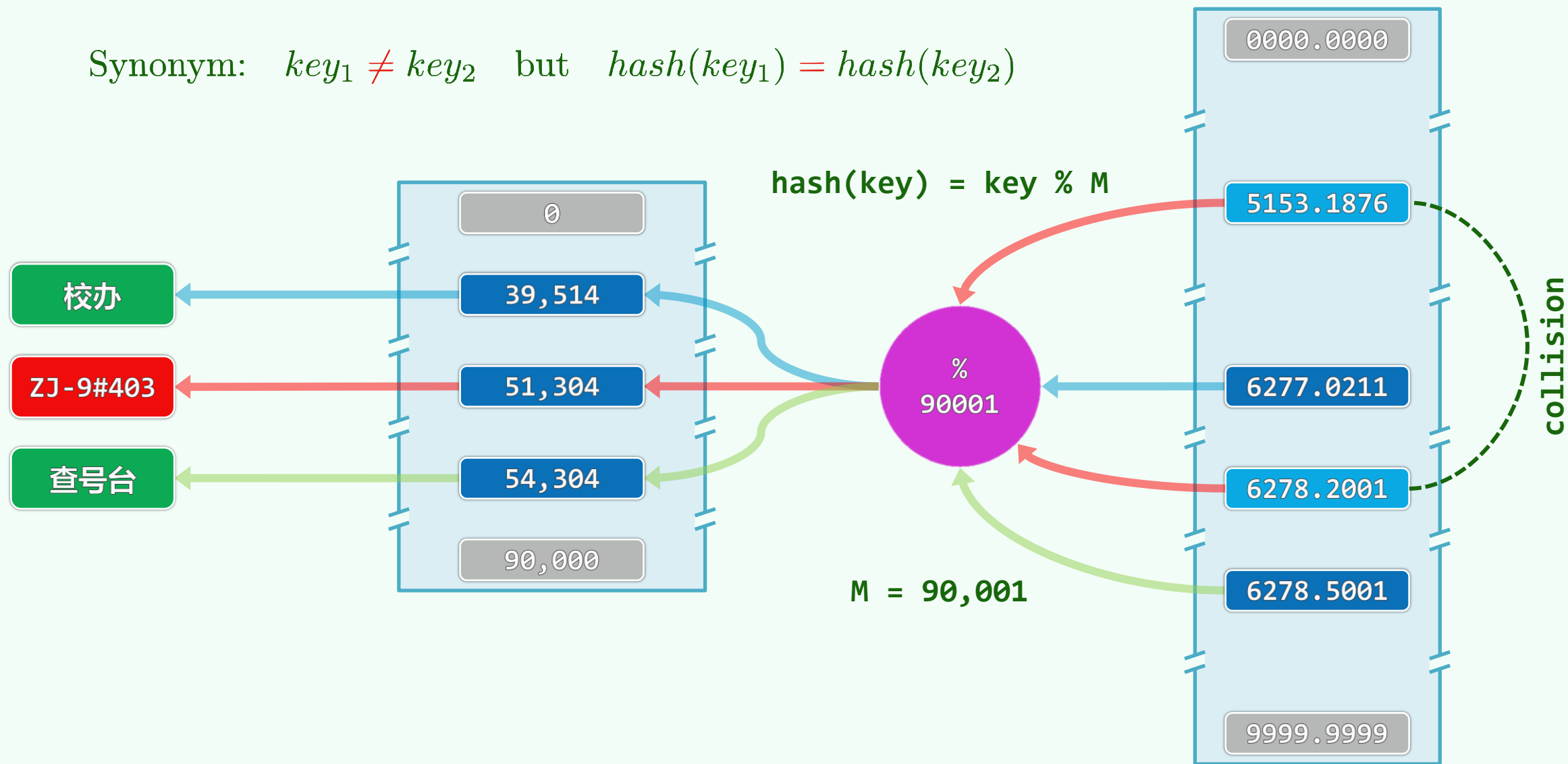
邓俊辉

deng@tsinghua.edu.cn

這是他來給你拜壽。今兒也是他的生日，你也該給他拜壽

同义词

Synonym: $key_1 \neq key_2$ but $hash(key_1) = hash(key_2)$

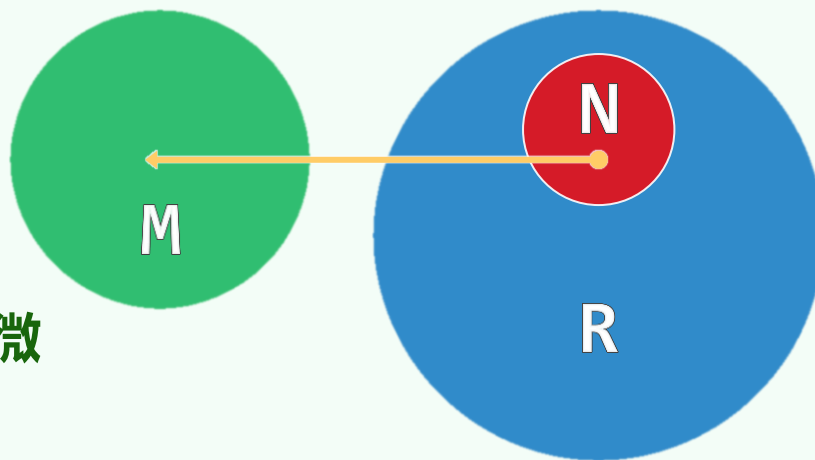


冲突 & 完美

❖ 装填因子 | load factor: $\lambda = \mathcal{N}/\mathcal{M}$

❖ λ 越大/小

- 空间利用率越高/低
 λ
- 冲突的情况越严重/轻微



❖ 通过降低 ^{λ} ，冲突程度将会有所改善

但只要数据集在动态变化，就无法彻底杜绝...

❖ 在特定条件下，的确可能实现单射 | injection

❖ 比如，当数据集已知且固定时，可实现

完美散列 | perfect hashing

- 采用两级散列模式
- 仅需 $\mathcal{O}(n)$ 空间
- 关键码之间互不冲突
- 最坏情况下也仅需 $\mathcal{O}(1)$ 时间

❖ 不过在一般情况下，完美散列可期不可求

必须面对现实，做好预案

生日悖论

❖ 将在座同学（对应的词条）按生日（月/日）做散列存储

散列表长 $M = 365$ ，装填因子 = 在场人数 $N / 365$

❖ 冲突（至少有两位同学生日相同）的可能性 $P_{365}(n) = ?$

$$P_{365}(1) = 0, P_{365}(2) = 1/365, \dots, P_{365}(22) = 47.6\%, P_{365}(23) = 50.7\%, \dots$$

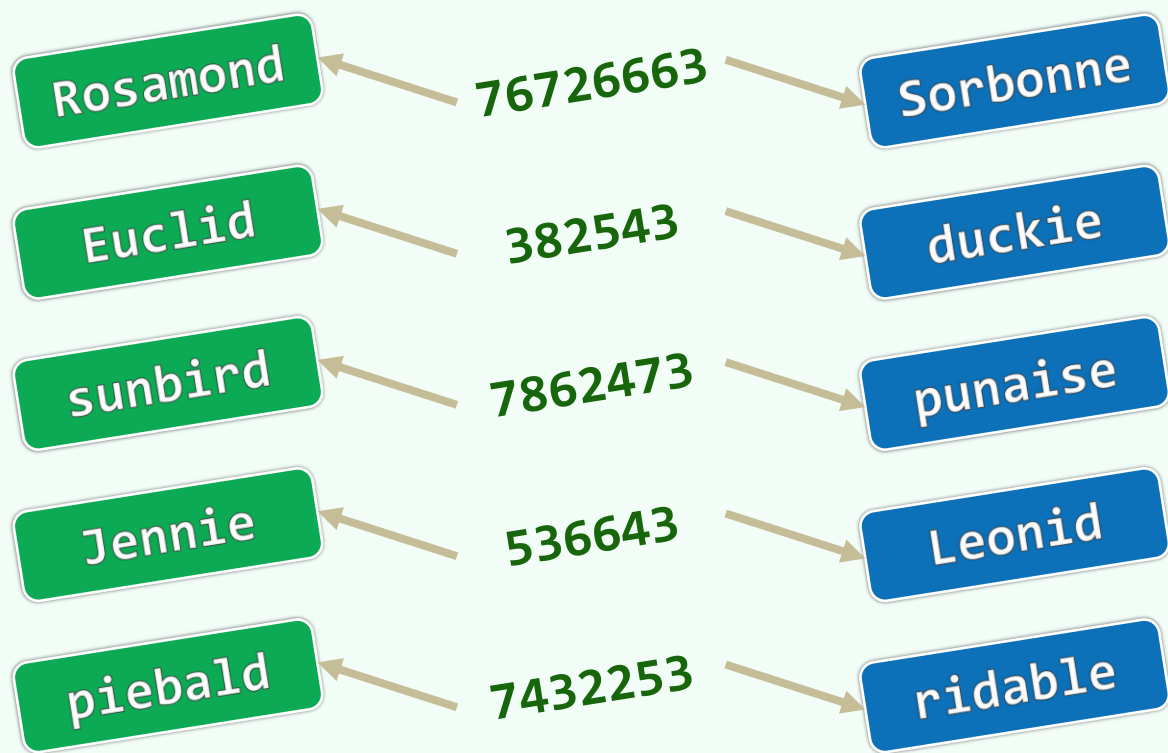
❖ 100人的集会： $1 - p_{365}(100) = 0.000,031\%$

- 自7岁起，不吃不喝、无休无息，每小时参加四次
- 到100岁，才有可能期望遇到一次无冲突的集会

❖ 因此，在装填因子确定之后，散列策略的选取将至关重要，散列函数的设计也很有讲究...

两项基本任务

- ❖ 首先（下一节）：精心**设计**散列表及散列函数，尽可能**降低**冲突的概率
- ❖ 同时（再下节）：制定可行的**预案**，以便在发生冲突时，能够尽快予以**排解**



词典

散列函数：基本

09-B1

此刻他就在占卜，方法是要从办公室到法庭扶手椅座位的步数可以被三除尽，那么新的疗法肯定能治好他的胃炎；要是除不尽，那就治不好。走下来是二十六步，但他把最后一步缩小，这样就正好走了二十七步

邓俊辉

deng@tsinghua.edu.cn

评价标准 + 设计原则

❖ 确定|determinism

同一关键码总是被映射至同一地址

❖ 快速|efficiency

expected- $O(1)$

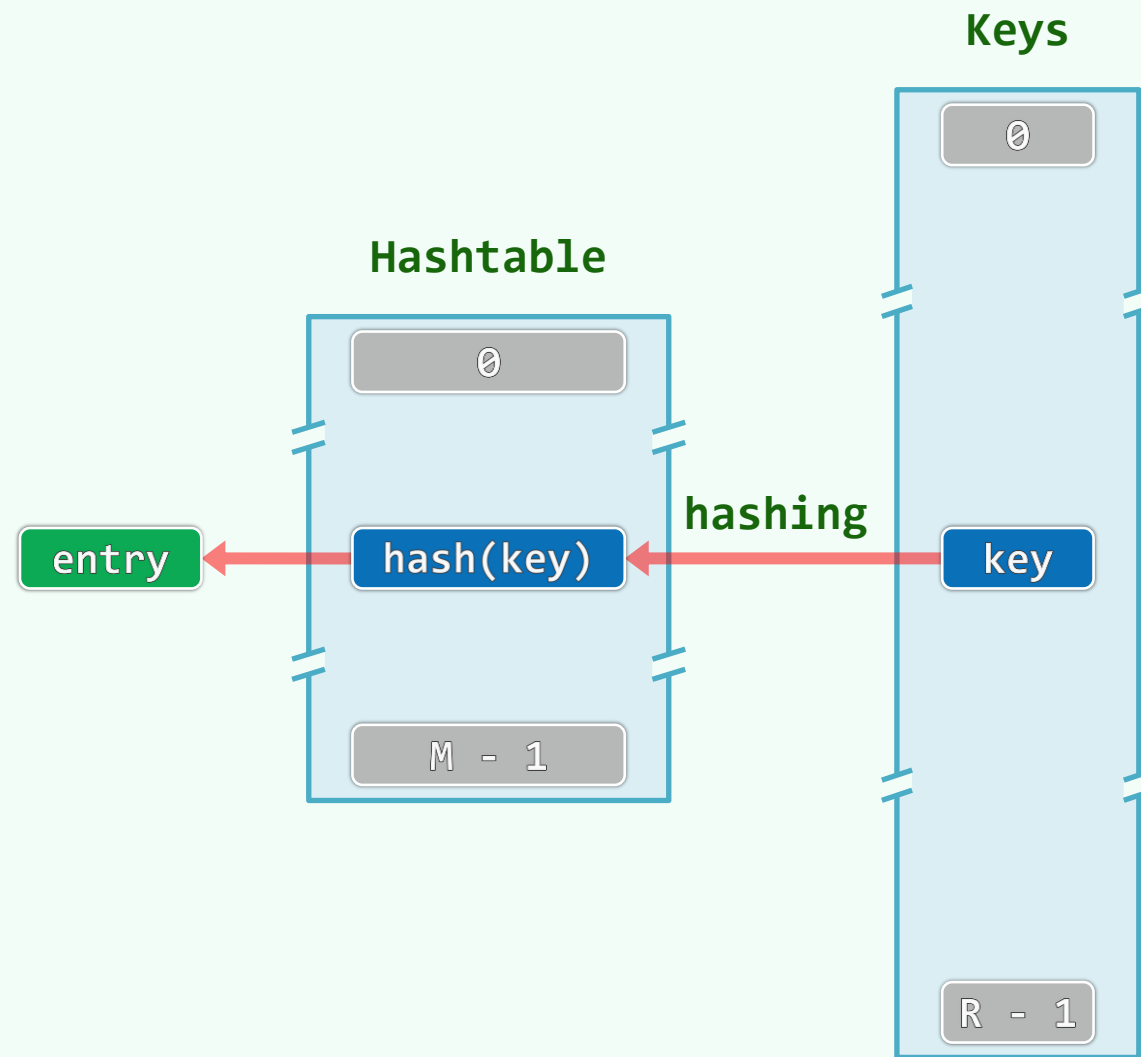
❖ 满射|surjection

尽可能充分地利用整个散列空间

❖ 均匀|uniformity

关键码映射到散列表各位置的概率尽量接近

有效避免数据的聚集|clustering



除余法

❖ $hash(key) = key \% M$ //前例中, 为何选 $M = 90001$?

❖ 据说: M 为**素数**时, 数据对散列表的覆盖最**充分**, 分布最**均匀**

其实: 对于**理想**随机的序列, 表长是否素数, **无关紧要**!

❖ 序列的**Kolmogorov**复杂度: 生成该序列的算法, 最短可用**多少行**代码实现?

- 算术级数: 7 12 17 22 27 32 37 42 47 ... //**单调性** / 线性: 从7开始, 步长为5
- 循环级数: 1 2 3 4 5 1 2 3 4 5 1 2 3 4 5 ... //**周期性**: 12345不断循环
- 英文: data structures and algorithms ... //**局部性**: 频率、关联、词根、...

❖ 实际应用中的数据序列**远非**理想随机, 上述规律性普遍地存在: Next-Token Prediction

蝉的哲学

❖ 面对往往具有周期的天敌/词条，将生命期/散列表长取作素数，可将聚集之概率降至最低

60			78			66			84			72		
30			48			36			54			42		
0			18			6			24			12		
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

$$\text{lcm}(M=15, 6) = 30$$
$$\text{gcd}(M=15, 6) = 3$$

48		66		84		54		72		90		60		78	
0		18		36		6		24		42		12		30	
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

$$\text{lcm}(M=16, 6) = 48$$
$$\text{gcd}(M=16, 6) = 2$$

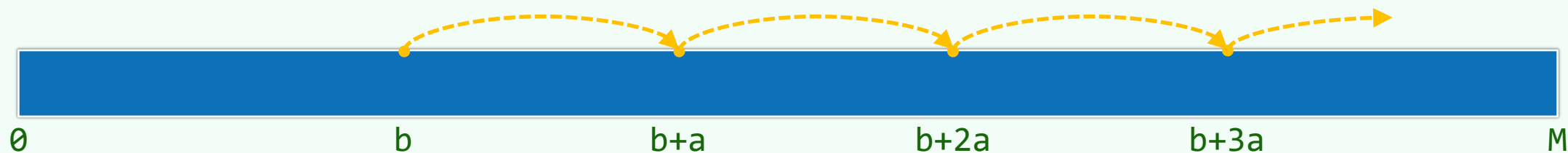
0	18	36	54	72	90	6	24	42	60	78	96	12	30	48	66	84
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

$$\text{lcm}(M=17, 6) = 102$$
$$\text{gcd}(M=17, 6) = 1$$

MAD法

❖ 除余法的缺陷

- **不动点**: 无论表长 M 取值如何, 总有: $hash(0) \equiv 0$
- **相关性**: $[0, R)$ 的关键码尽管系平均分配至 M 个桶; 但**相邻**关键码的散列地址也必**相邻**



❖ Multiply-Add-Divide

$$hash(key) = (a \times key + b) \% M, \quad M \text{ prime}, \quad a > 1, \quad b > 0, \quad \text{and } M \nmid a$$

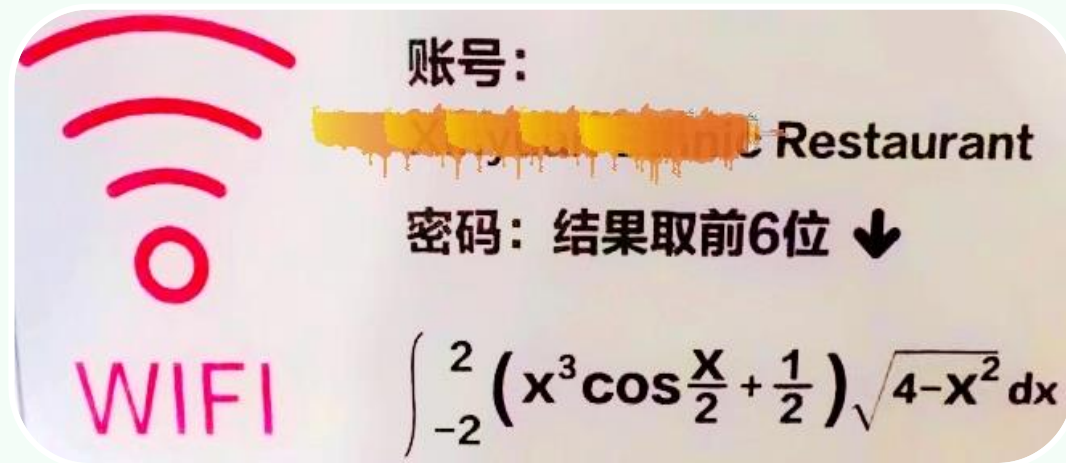
更多散列函数

❖ 数字分析/selecting digits

抽取key中的某几位，构成地址

- 比如，取十进制表示的奇数位

$$\text{hash}(\overset{3}{1} \overset{4}{1} \overset{5}{9} \overset{2}{6} \overset{5}{3} \overset{5}{9}) = 345255$$



❖ 平方取中/mid-square

取key²的中间若干位，构成地址

$$\text{hash}(123) = \text{middle}(123 \times 123) = 1\boxed{512}9 = 512$$

$$\text{hash}(1234567) = 15241\boxed{556}77489 = 556$$



更多散列函数

❖ 折叠法/folding: 将key分割成等宽的若干段, 取其总和作为地址

$$\text{- hash}(123^{456}789) = 123 + 456 + 789 = 1368 \quad // \text{自左向右}$$

$$\text{- hash}(123^{456}789) = 123 + 654 + 789 = 1566 \quad // \text{往复折返}$$

❖ 位异或法/XOR: 将key分割成等宽的二进制段, 经异或运算得到地址

$$\text{- hash}(110^{011}011_b) = 110 \wedge 011 \wedge 011 = 110_b \quad // \text{自左向右}$$

$$\text{- hash}(110^{011}011_b) = 110 \wedge 110 \wedge 011 = 011_b \quad // \text{往复折返}$$

❖ ... 总之, 越是随机, 越是没有规律, 越好

散列

散列函数：随机数

又由谁去体现这世间的幸福、骄傲和快乐？只好听凭偶然，是没有道理好讲的

不好，不好！我這裡有一方手帕，你頂在頭上，遮了臉，撞個天婚，
教我女兒從你跟前走過，你伸開手扯倒那個就把那個配了你罷

天下就没有偶然，那不过是化了妆的、戴了面具的必然

邓俊辉

deng@tsinghua.edu.cn

(伪) 随机数法

❖ 循环: $\text{rand}(x + 1) = [a \times \text{rand}(x)] \% M$ // M 素数, $a \% M \neq 0$

$$a = 7^5 = 16,807 = \boxed{100000110100111}_b$$

$$M = 2^{31} - 1 = 2,147,483,647 = 01111111 \boxed{11111111} 11111111 \boxed{11111111}_b$$

❖ 径取: $\text{hash}(\text{key}) = \text{rand}(\text{key}) = [\text{rand}(0) \times a^{\text{key}}] \% M$

种子: $\text{rand}(0) = ?$

❖ 把难题推给伪随机数发生器, 但是...

❖ (伪) 随机数发生器的实现, 因具体平台、不同历史版本而异

创建的散列表可移植性差——故需慎用此法!

(伪) 随机数法: The C Programming Language (2nd edn), p46

❖ `unsigned long int next = 1; //sizeof(long int) = 8`

`void srand( unsigned int seed ) { next = seed; } //sizeof(int) = 4 or 8`

`int rand( void ) { //1103515245 =  $3^5 * 5 * 7 * 129749$`

`next = next * 1103515245 + 12345; //M + A + ...`

`return (unsigned int) (next/65536) % 32768; //...D`

`}`

next



$M + A$

rand



❖ `int rand() { int uninitialized; return uninitialized; }`

`char* rand( t_size n ) { return ( char* ) malloc( n ); }`

就地随机置乱：任给一个数组A[0, n), 理想地将其中元素的次序随机打乱

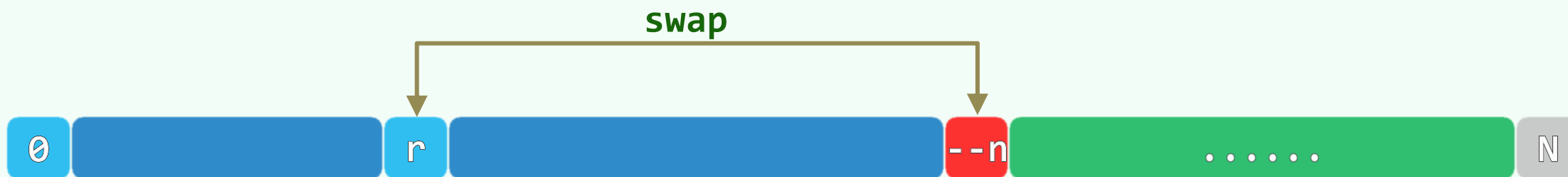
//[R. Fisher & F. Yates, 1938], [R. Durstenfeld, 1964], [D. E. Knuth, 1969]

```
void shuffle( int A[], int n ) {
```

```
    for ( ; 1 < n; --n ) //自后向前, 依次将各元素
```

```
        swap( A[ rand() % n ], A[ n - 1 ] ); //与随机选取的某一前驱 (含自身) 交换
```

```
    } //20! < 2^64 < 21!
```



❖ 的确可以等概率地生成所有 $n!$ 种排列? $20! < 2^{64} < 21!$

词典

散列函数：hashCode与多项式法

有意整齊與有意變化，皆是一方死法

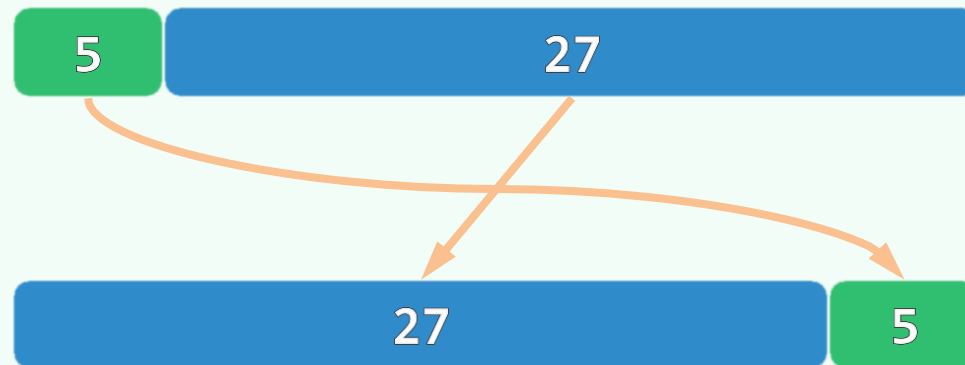
于是父亲只得求助于拈阄的办法，把两个姑娘的姓氏写在两方小红纸片上，把它们揉成两团，拿在手里，走到祖宗的神主面前诚心祷告了一番，然后随意拈起一个来。李家的亲事就这样地决定了

邓俊辉

deng@tsinghua.edu.cn

String/Object To Integer

```
static Rank hashCode( char s[] ) {  
    Rank n = strlen(s); Rank h = 0;  
    for ( Rank i = 0; i < n; i++ ) {  
        h = (h << 5) | (h >> 27);  
        h += s[i];  
    } //乘以32~27, 加上扰动, 累计贡献  
    return h;  
}  
//有必要如此复杂吗? 能否使用更简单的散列, 比如...
```



$$\begin{aligned} & \text{hashCode}("x_{n-1} \dots x_3 x_2 x_1 x_0") \\ &= x_{n-1} \cdot a^{n-1} + \dots + x_2 \cdot a^2 + x_1 \cdot a^1 + x_0 \\ &= (\dots ((x_{n-1} \cdot a + x_{n-2}) \cdot a) + \dots + x_1) \cdot a + x_0 \end{aligned}$$

冲突 ~ 巧合

❖ 比如: $hashCode(S) = \sum_{c \in S} code(upper(c))$

$$hashCode("hash") = 8 + 1 + 19 + 8 = 36$$

❖ 字符相对次序信息丢失, 将引发大量冲突

- I am Lord Voldemort
- Tom Marvolo Riddle

❖ 即便字符不同、数目不等...

- He's Harry Potter

❖ Key to improving your programming skills

Learning Tsinghua Data Structures & Algorithms



词典

排解冲突：开放散列

09-C1

盡人事，知天命

Every mistake I've ever made
Has been rehashed and then replayed
As I got lost along the way.

邓俊辉

deng@tsinghua.edu.cn

多槽位/Multiple Slots

❖ 桶单元细分成若干槽位

存放（与同一单元）冲突的词条

❖ 只要槽位数目不太多

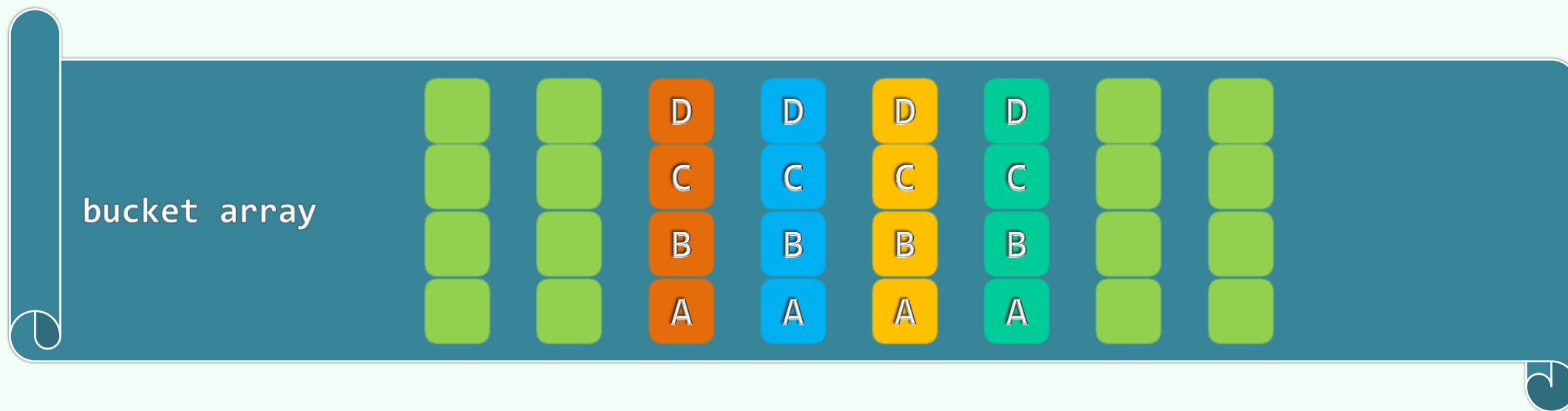
依然可以保证 $O(1)$ 的时间效率

❖ 但是，究竟需要细分到什么程度？

❖ 难以预测！

- 过细，空间浪费；反过来

- 无论多细，极端情况下仍可能不够



独立链 / Linked-List Chaining / Separate Chaining

❖ 每个桶拥有一个列表，存放对应的一组同义词

❖ 优点 无需为每个桶预备多个槽位

任意多次的冲突都可解决

删除操作实现简单、统一

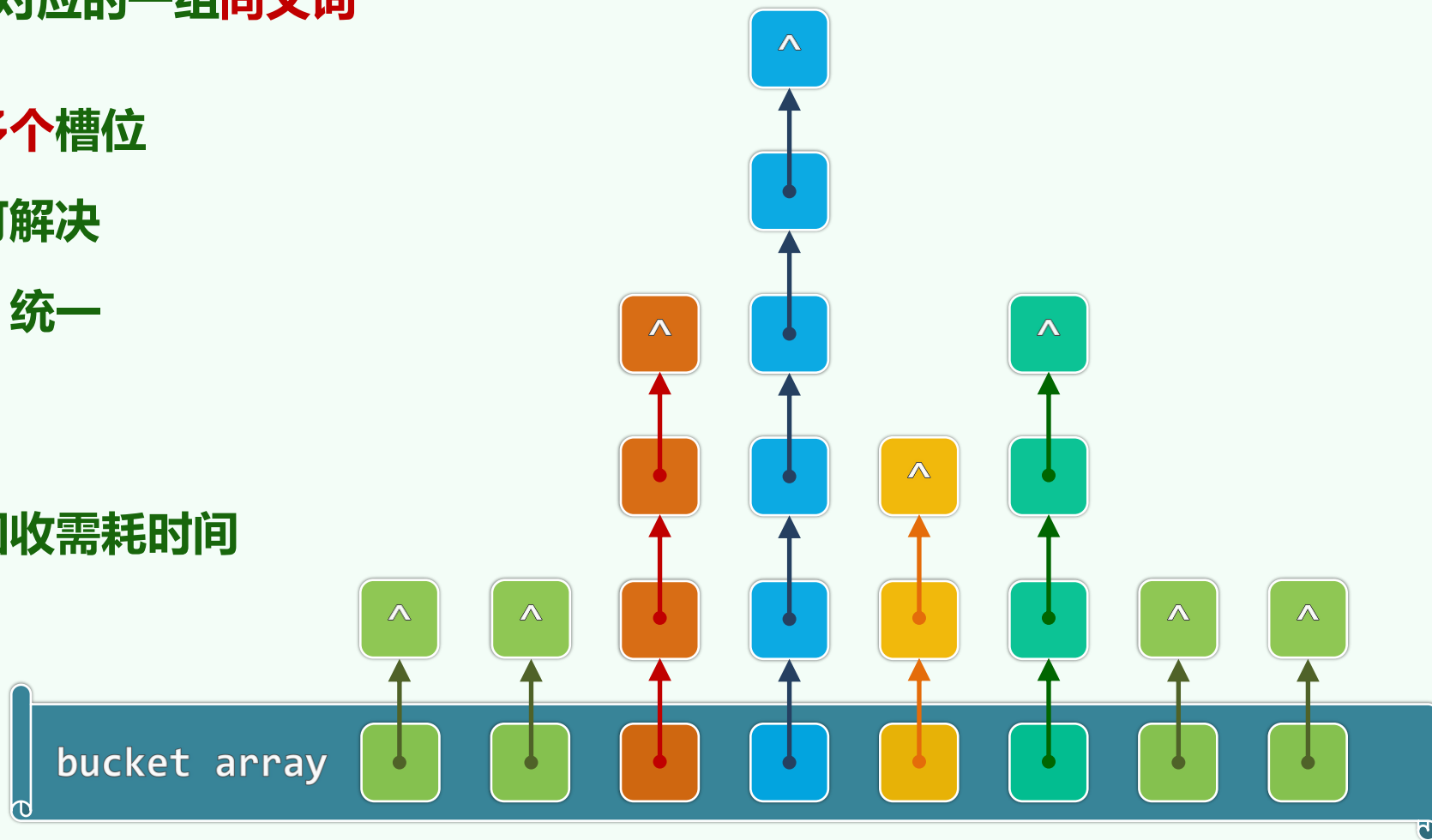
❖ 但是 指针本身占用空间

节点的动态分配和回收需耗时间

而更重要的是...

❖ 空间未必连续分布

系统缓存很难生效



词典

排解冲突：封闭散列

旅客要在每个生人门口敲叩 才能敲到自己的家门
人要在外面到处漂流 最后才能走到最深的内殿

在我们出生之前，一切都在没有我们的宇宙里开着
在我们活着的时候，一切都在我们身体里闭着
当我们死去，一切重又打开
打开、关闭、打开，我们就是这样

一俊遮百丑

邓俊辉

deng@tsinghua.edu.cn

公共溢出区 / Overflow Area

❖ 单独开辟一块**连续**空间

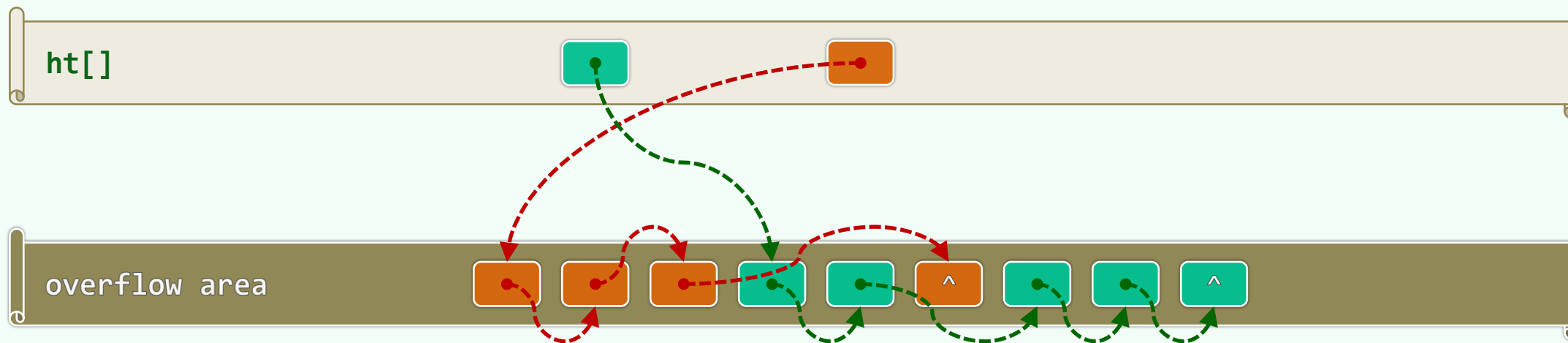
发生冲突的词条，**顺序**存入此区域

❖ 结构简单，算法易于实现

❖ 但是，不冲突则已，**一旦**发生冲突...

❖ 最坏情况下，处理冲突词条所需的时间将

正比于溢出区的**规模**

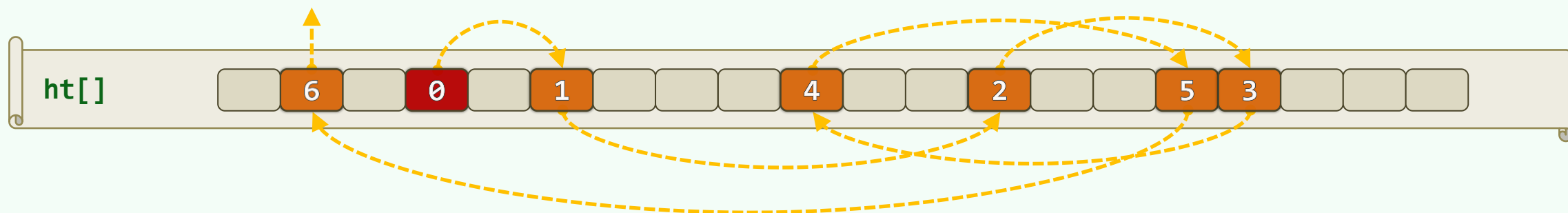


封闭散列 x 开放定址 x 试探链

❖ Closed Hashing: 散列表的物理空间相对**固定**；所有冲突都在**内部**寻求解决

Open Addressing: 只要有**必要**，任何散列桶**都可以**接纳任何词条

❖ 与开放散列恰好**相反**，整体结构更加整饬、简明，cache机制可大显身手



❖ Probe Sequence/Chain: 为每一组同义词，都**事先约定**了若干**备用桶**，依次串接成序列/链

❖ 查找算法：沿**试探链**逐个检视**下一桶单元**，直到命中**成功**，或者抵达一个**空桶而失败**

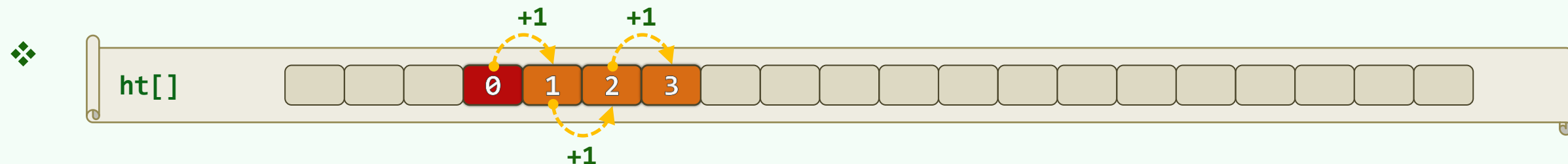
❖ 具体地，试探链又应如何**约定**？

线性试探

❖ **Linear Probing:** $r_i(key) = (\text{hash}(key) + i) \bmod \mathcal{M}, i = 0, 1, 2, 3, \dots$

❖ 当然，只要还有**空桶**，试探链就不致**循环**，查找必能**终止**

然而，试探链有相互**重叠**，**非同义词**之间也会彼此**堆积** (clustering)



❖ 但只要**控制好装填因子**，冲突与堆积都**不致太严重**：

$$\begin{aligned}\mathbb{E}(\text{probes}) &= \sum_{k=1}^{\infty} k \cdot \lambda^{k-1} \cdot (1 - \lambda) = (1 - \lambda) \cdot \sum_{k=1}^{\infty} k \cdot \lambda^{k-1} = (1 - \lambda) \cdot \sum_{k=1}^{\infty} (\lambda^k)' \\ &= (1 - \lambda) \cdot \left(\sum_{k=1}^{\infty} \lambda^k \right)' = (1 - \lambda) \cdot (1/(1 - \lambda))' = (1 - \lambda) \cdot 1/(1 - \lambda)^2 = 1/(1 - \lambda)\end{aligned}$$

❖ 更重要地，**一俊遮百丑**...这是典型的**Stride-1 Reference Pattern**

数据局部性极佳，系统**缓存**的作用可发挥到**极致**

词典

排解冲突：懒惰删除

09-C3

邓俊辉

deng@tsinghua.edu.cn

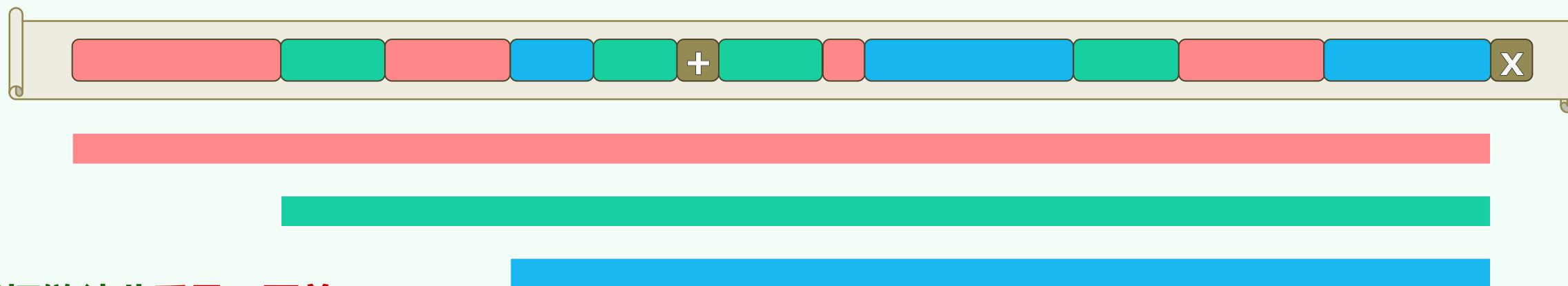
斜陽草樹，尋常巷陌，人道寄奴曾住

江山故宅空文藻，雲雨荒台豈夢思
最是楚宮俱泯滅，舟人指點到今疑

插入 + 删除

❖ 插入：新词条若尚不存在，便可存入试探**终止**处的空桶

❖ 删除：简单地将命中的桶**清空**？不可！



❖ 试探链彼此**重叠、覆盖**！

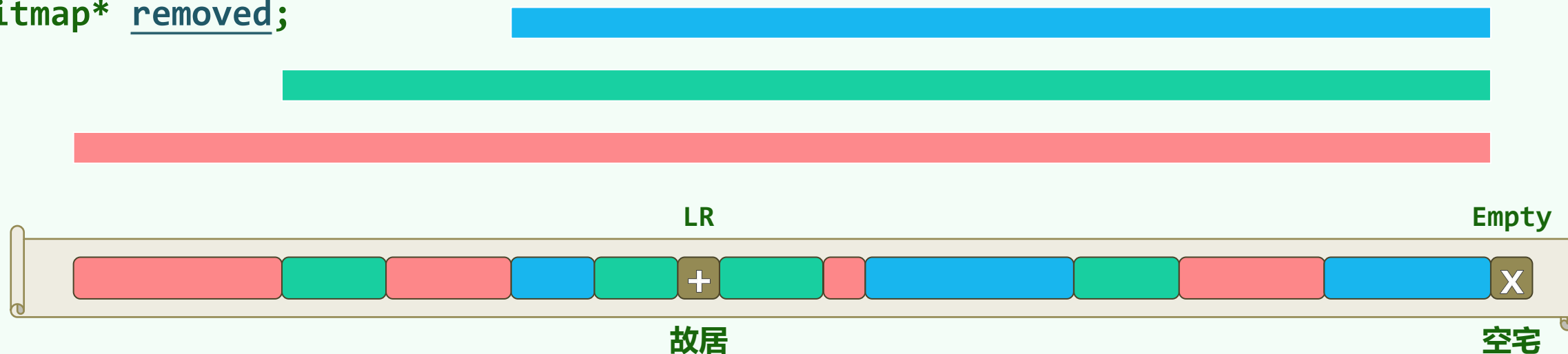
一个桶的清空，将令**多条**试探链在此**断裂**，导致后续的词条**丢失**——明明存在，却访问不到

❖ 那么，如何才能**简明、高效而安全**地完成删除呢？

Lazy Removal

❖ 仅做**标记**，不对试探链做更多调整：

Bitmap* removed;

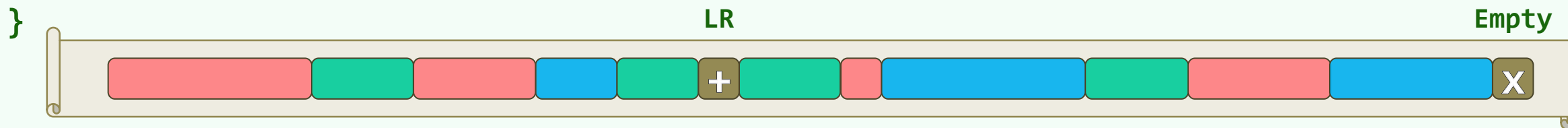


❖ 此后，带标记桶的**角色**因具体的操作而异（装填因子的计算**口径**亦有区别）

- **查找**词条时，被视作“**必不匹配的非空桶**”，试探链在此得以**延续**
- **插入**词条时，被视作“**总是匹配的空闲桶**”，可以用来**存放**新词条

两种试探

```
template <typename K, typename V> Rank Hashtable<K, V>::probe4Hit( const K & k ) {  
    for ( Rank r = hashCode(k) % M; ; r = (r+1) % M ) //按除余法确定起点, 线性试探  
        if ( ht[r] && (k==ht[r]->key) || !ht[r] && !removed->test(r) ) //跳过懒惰删除的桶  
            return r; //只要N+L < M, 迟早能终止  
}
```



```
template <typename K, typename V> Rank Hashtable<K, V>::probe4Free( const K & k ) {  
    for ( Rank r = hashCode(k) % M; ; r = (r+1) % M ) //按除余法确定起点, 线性试探  
        if ( !ht[r] ) //只要有空桶 (无论是否带有懒惰删除标志)  
            return r; //迟早能找到  
}
```

09-C4

词典

排解冲突：重散列

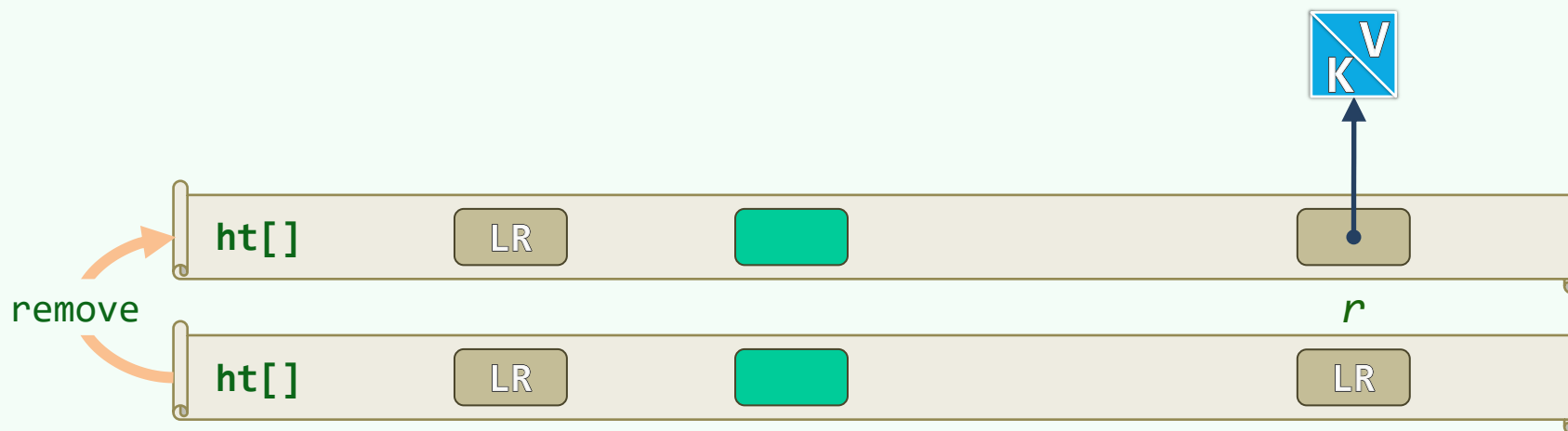
天地會壞否？不會壞。只是相將人無道極了，
便一齊打合，混沌一番，人物都盡，又重新起

邓俊辉

deng@tsinghua.edu.cn

删除

```
template <typename K, typename V> bool Hashtable<K, V>::remove( K k ) {  
  
    Rank r = probe4Hit( k ); if ( !ht[r] ) return false; //确认目标词条确实存在  
  
    delete ht[r]; ht[r] = NULL; --N; removed->set(r); //清除词条, 设置LR标记  
  
    return true;  
  
}
```



插入

```
template <typename K, typename V> Entry<K, V>* Hashtable<K, V>::put( K k, V v ) {  
  
    Rank r = probe4Free( k ); //找个空桶 (只要装填因子控制得当, 必然成功)  
  
    ht[r] = new Entry<K, V>( k, v ); removed->clear( r ); //插入词条, 清除LR标记  
  
    ++N; if ( ( N + removed->size() ) * 2 > M ) rehash(); //若装填因子高于50%, 重散列  
  
    return ht[r]; //如禁止词条相等, 调用前需先get()确认  
}
```



Rehashing: 随着装填因子攀升, 冲突激增; 超过阈值后, 便需要"扩"容

```
template <typename K, typename V> void Hashtable<K, V>::rehash() {
```

```
    Rank M0 = M; M = primeNLT( 4 * N ); //新的容量
```

```
    Entry<K, V>** ht0 = ht; ht = new Entry<K, V>*[ M ]; //极端情况下, 反而会缩容
```

```
    memset( ht, 0, sizeof( Entry<K, V>* ) * M ); //初始清空
```

```
    for ( Rank i = 0; i < M0; i++ ) //遍历原表
```

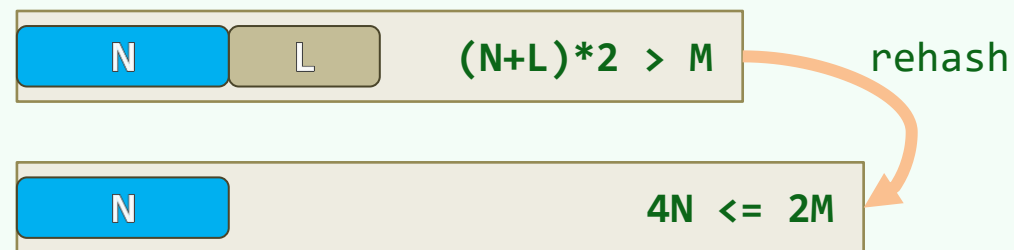
```
        if ( ht0[i] ) { //每个词条
```

```
            ht[ probe4Free( ht0[i]->key ) ] = ht0[i]; //逐个转入新表
```

```
    delete removed; removed = new Bitmap( M ); //懒惰删除标记
```

```
    delete[] oldHt; //原表至此已空, 释放之
```

```
}
```



词典

排解冲突：平方试探

09-C5

邓俊辉

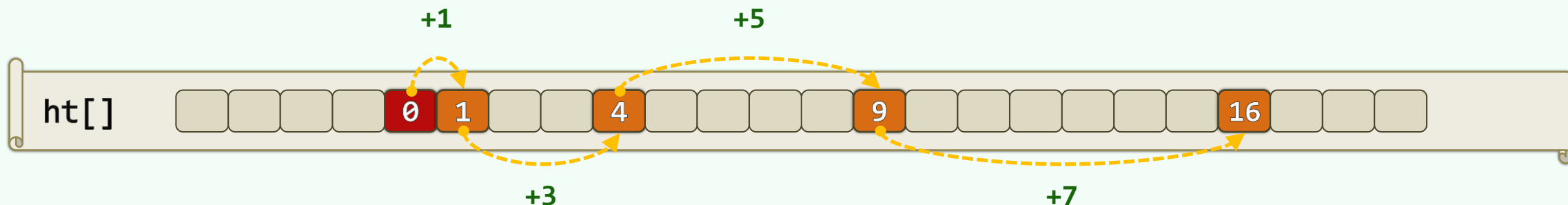
deng@tsinghua.edu.cn

三十六計，走為上計

我真的以为，这样何尝不是一种所谓的解脱
要背负的辛苦又有谁能够清楚，那内心的冲突

平方试探

❖ Quadratic Probing: $r_i(key) = (\text{hash}(key) + i^2) \bmod \mathcal{M}$, $i = 0, 1, 2, 3, \dots$



❖ 沿着试探链，各桶的间距**线性**递增：一旦冲突，可“**聪明**”地跳离是非之地

❖ 数据**堆积**现象有所改善

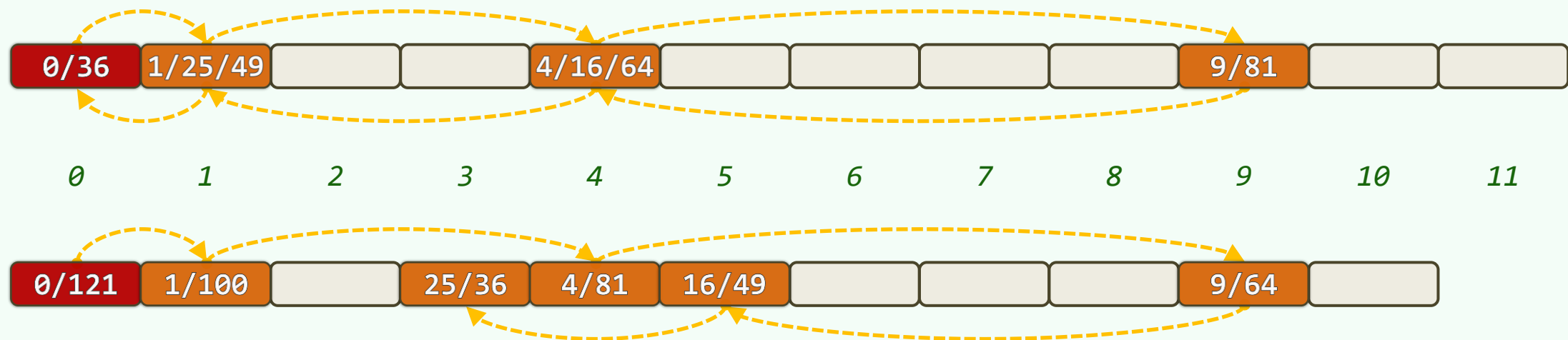
❖ **cache**的利用率有所下降，好在还不算差

❖ 只要有**空桶**，就...一定能...找到吗？毕竟，可不再是**逐个**地试探了

素数表长时，只要 $\lambda < 0.5$ 就一定能够找出；否则，不见得

❖ $\{0, 1, 2, 3, 4, 5, \dots\}^2 \% 12 = \{0, 1, 4, 9\}$

❖ M 若为合数， $n^2 \% M$ 可能的取值可能少于 $\lceil M/2 \rceil$ 种——此时，只要对应的桶均非空...



❖ Quadratic Residue: 若 M 为素数，则 $n^2 \% M$ 恰有 $\lceil M/2 \rceil$ 种取值，且由试探链的前 $\lceil M/2 \rceil$ 项取遍

❖ $\{0, 1, 2, 3, 4, 5, \dots\}^2 \% 11 = \{0, 1, 4, 9, 5, 3\}$

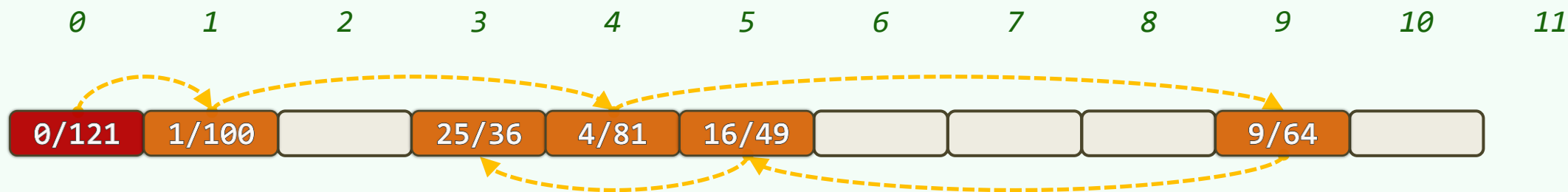
每一条试探链，都有一个足够长的无重前缀

❖ 反证：假设存在 $0 \leq a < b < \lceil \mathcal{M}/2 \rceil$ ，使得沿着试探链，第 a 项和第 b 项彼此冲突

❖ 于是： a^2 和 b^2 自然关于 $\mathcal{M}$ 同余，亦即 $a^2 \equiv b^2 \pmod{\mathcal{M}}$

$$b^2 - a^2 = (b + a) \cdot (b - a) \equiv 0 \pmod{\mathcal{M}}$$

❖ 然而， $0 < b - a \leq b + a < \lceil \mathcal{M}/2 \rceil + (\lceil \mathcal{M}/2 \rceil - 1) \leq \lceil \mathcal{M}/2 \rceil + \lfloor \mathcal{M}/2 \rfloor = \mathcal{M}$



... 无论 $b - a$ 还是 $b + a$ 都不可能整除 $\mathcal{M}$ [QED]

❖ 那么，另一半的桶，可否也利用起来呢...

词典

排解冲突：双向平方试探

09-C6

繞樹三匝，何枝可依

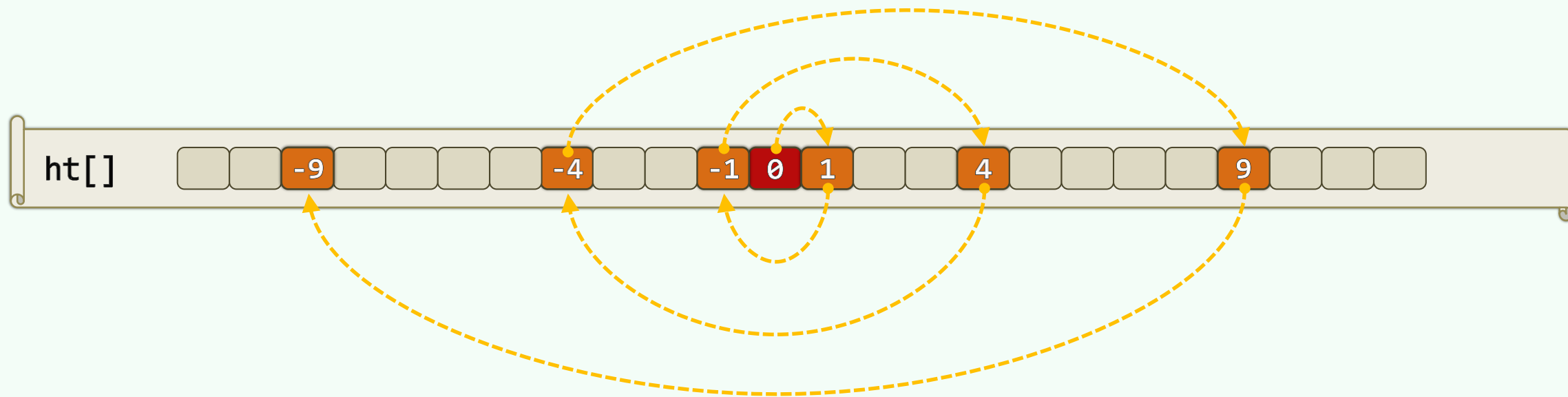
伺候父亲躺下，我正准备离去，父亲拉住了我的手，轻轻地问我，
丫儿，你知道什么是无枝可栖吗？

邓俊辉

deng@tsinghua.edu.cn

策略：交替地沿两个方向试探，均按平方确定距离

$$r_i(key) = (\text{hash}(key) + (-1)^{i+1} \cdot (\lceil i/2 \rceil)^2) \bmod \mathcal{M}, \quad i = 0, 1, 2, 3, \dots$$



子试探链，彼此独立？

❖ 正向和反向的子试探链，各自包含 $\lceil \mathcal{M}/2 \rceil$ 个互异的桶

$$\underbrace{-\lceil \mathcal{M}/2 \rceil, \dots, -3, -2, -1, 0, 1, 2, 3, \dots, \lfloor \mathcal{M}/2 \rfloor}$$

$\pm i^2$		-36	-25	-16	-9	-4	-1	0	1	4	9	16	25	36
M	5					1	4	0	1	4				
	7				5	3	6	0	1	4	2			
	11		8	6	2	7	10	0	1	4	9	5	3	
	13	3	1	10	4	9	12	0	1	4	9	3	12	10

❖ 除了起点0，这两个序列是否还有...其它公共的桶？

$$4k + 3$$

❖ 两类素数: $3 \quad 5 \quad 7 \quad 11 \quad 13 \quad 17 \quad 19 \quad 23 \quad 29 \quad 31$

❖ 诀窍: 表长取作素数 $M = 4 \cdot k + 3$, 即**必然可以保证**试探链的前 M 项均互异

$\pm i^2$		-36	-25	-16	-9	-4	-1	0	1	4	9	16	25	36
M	5					1	4	0	1	4				
	7				5	3	6	0	1	4	2			
	11		8	6	2	7	10	0	1	4	9	5	3	
	13	3	1	10	4	9	12	0	1	4	9	3	12	10

❖ 反之, $M = 4 \cdot k + 1$ 就...**必然不能使用?**

Two-Square Theorem of Fermat

❖ 素数 p **不能**表示为一对整数的平方和，当且仅当

$$p \equiv 3 \pmod{4}$$

❖ 只要注意到：

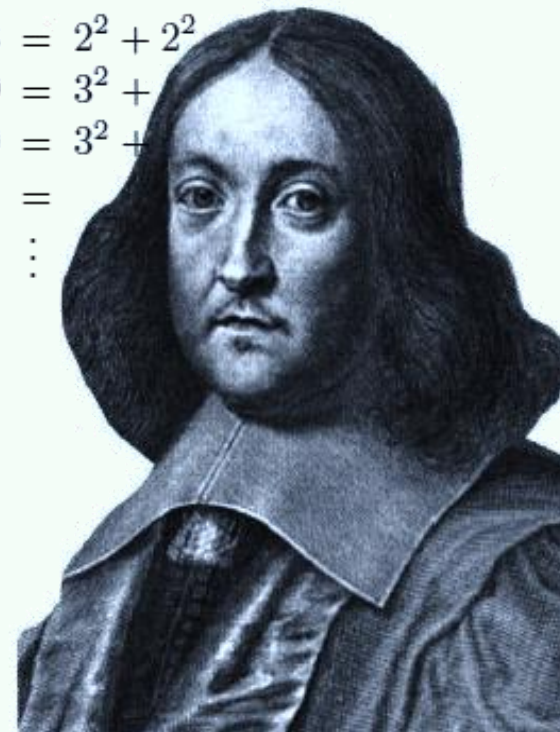
$$\begin{aligned}(u^2 + v^2) \cdot (s^2 + t^2) &= (us + vt)^2 + (ut - vs)^2 \\ &= (us - vt)^2 + (ut + vs)^2\end{aligned}$$

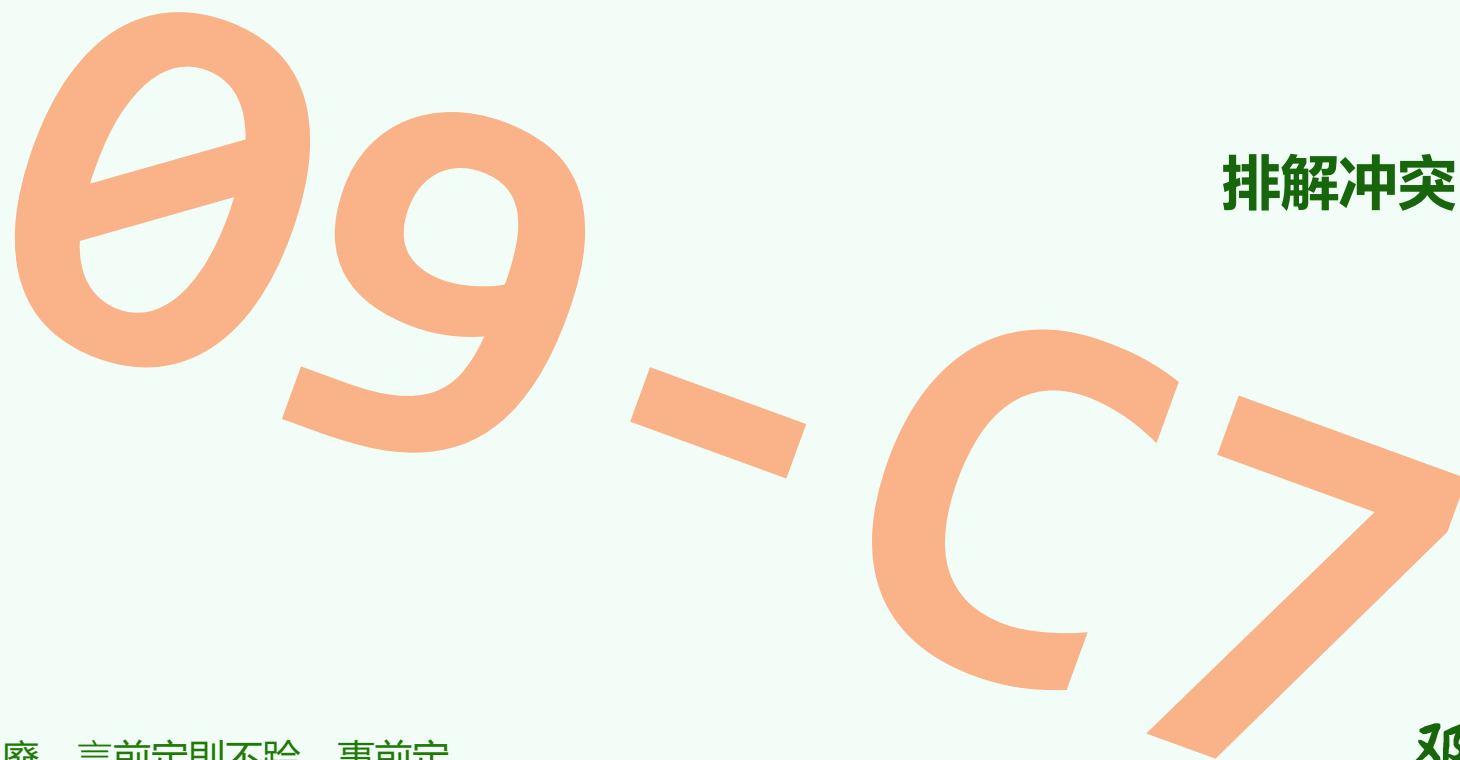
$$\begin{aligned}(2^2 + 3^2) \cdot (5^2 + 8^2) &= (10 + 24)^2 + (16 - 15)^2 \\ &= (10 - 24)^2 + (16 + 15)^2\end{aligned}$$

❖ 就可以推知：

- 自然数 n 可表示为一对整数的平方和，当（且**仅当**）
- 它的每一 $M = 4 \cdot k + 3$ 类的素因子均为**偶数**次方

$$\begin{aligned}1 &= 1^2 + 0^2 \\ 2 &= 1^2 + 1^2 \\ 3 &= \\ 4 &= 2^2 + 0^2 \\ 5 &= 2^2 + 1^2 \\ 6 &= \\ 7 &= \\ 8 &= 2^2 + 2^2 \\ 9 &= 3^2 + \\ 10 &= 3^2 + \\ 11 &= \\ &\vdots\end{aligned}$$





词典

排解冲突：双散列

凡事豫則立，不豫則廢。言前定則不跲，事前定
則不困，行前定則不疚，道前定則不窮

邓俊辉

deng@tsinghua.edu.cn

Double Hashing

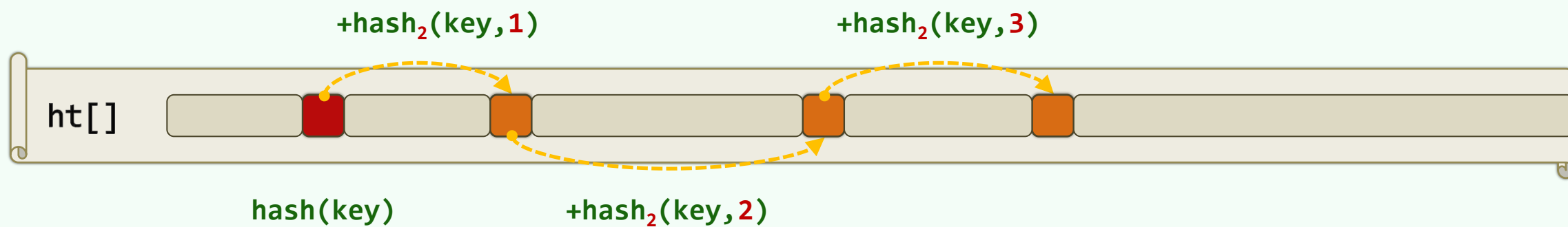
❖ 预先约定第二散列函数: $hash_2(key, i)$

冲突时, 由其确定偏移增量, 确定下一试探位置: $[hash(key) + \sum_{i=1}^k hash_2(key, i)] \% M$

❖ 线性试探: $hash_2(key, i) \equiv 1$

平方试探: $hash_2(key, i) = 2i - 1$

更一般地, 偏移增量同时还与key相关



词典

桶排序与最大缝隙

eg-D

Don't put all your eggs in one basket.

恒纪元能持续多长时间？

一天或一个世纪，每次多长谁都不准

那乱纪元会持续多长时间呢？

不是说过嘛，除了恒纪元都是乱纪元，两者互为对方的间隙

邓俊辉

deng@tsinghua.edu.cn

大数据 + 小集合

❖ $[0, m)$ 内 n 个互异整数, $m \ll n$

❖ 使用开放的散列表

相等元素 ~ 同义词

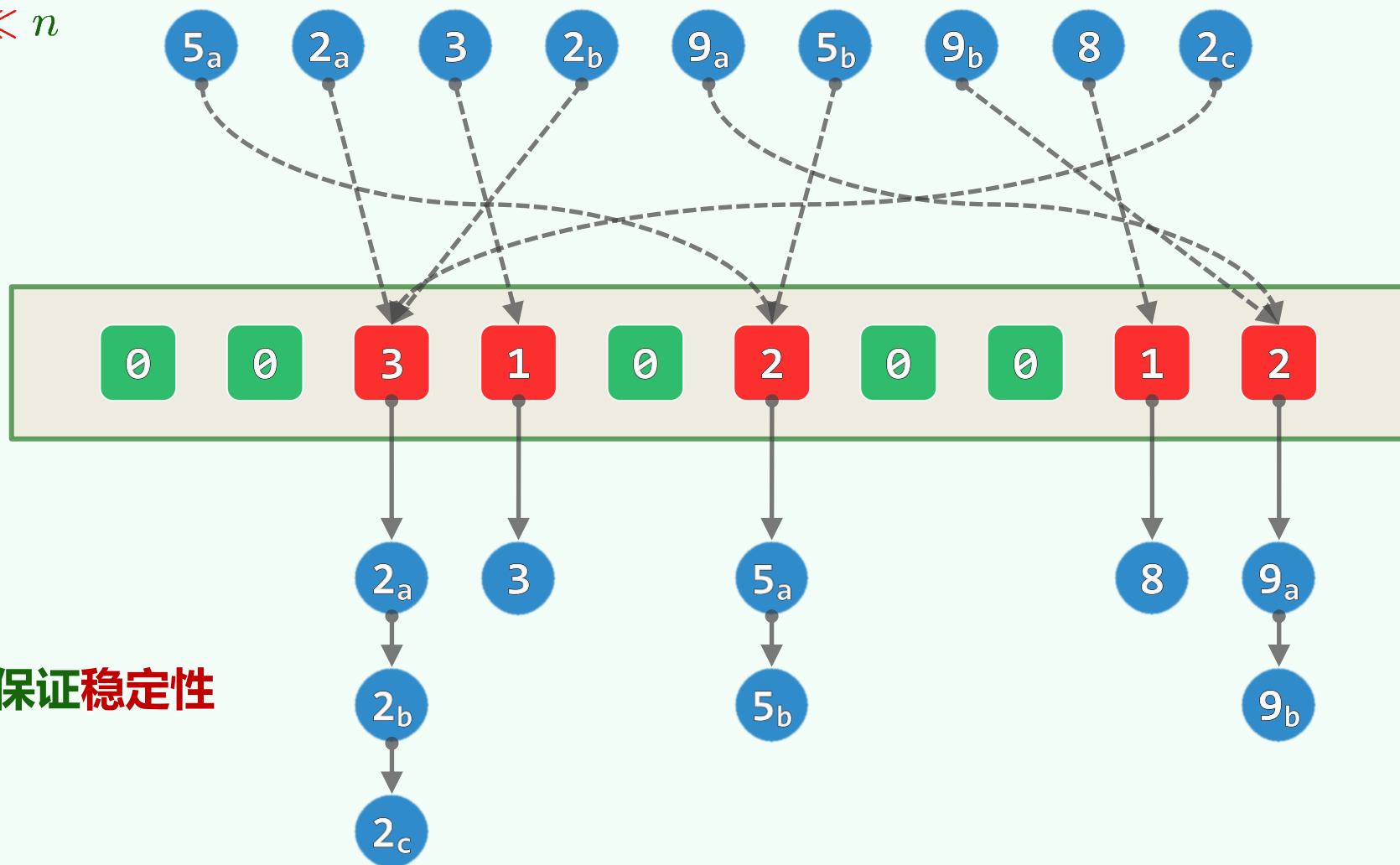
❖ Initialize: ~~$\mathcal{O}(m)$~~ $\mathcal{O}(1)$

Distribute: $\mathcal{O}(n)$

Collect: $\mathcal{O}(m)$

❖ 链表串接的次序得当, 便可保证稳定性

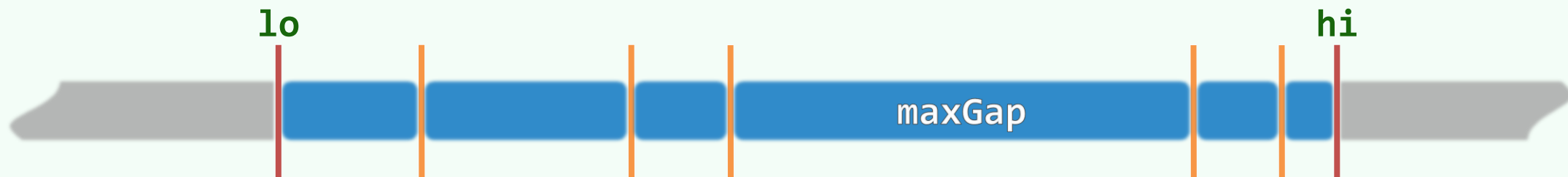
❖ $\mathcal{O}(m+n)$ 空间 + $\mathcal{O}(n)$ 时间



MaxGap

❖ 任意 n 个点，均将实轴分为至多 $n-1$ 段有界区间

其中的哪一段**最长**？



❖ 如果不讲究效率，显而易见的方法莫过于

- 对所有点**排序** // 可惜，这需要 $\Omega(n \log n)$ 时间
- 依次计算各相邻点对的间距，保留最大者 // $\Theta(n)$

❖ 可否更快？采用**分桶**策略，可改进至 $\mathcal{O}(n)$ 时间...

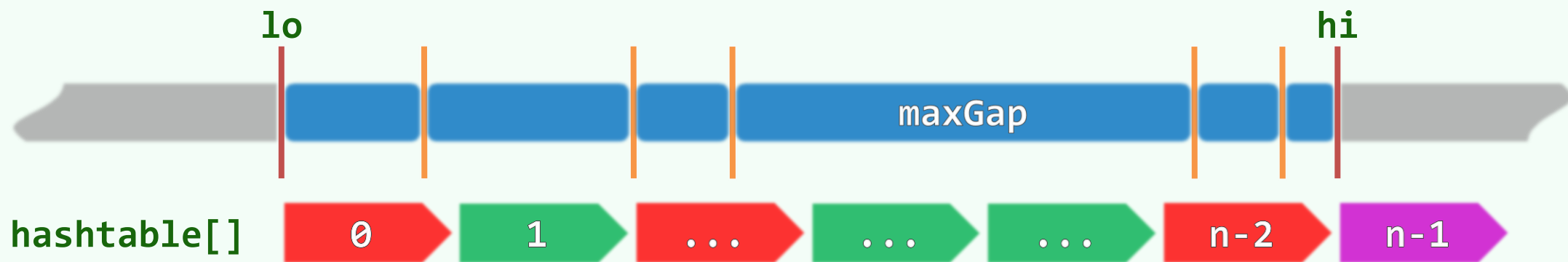
线性算法

找到全集的**最左点**、**最右点**

$O(n)$ //一趟线性扫描

将有效范围**均分**为 $n-1$ 段 (n 个桶)

$O(n)$ //相当于散列表



通过散列，将各点**归入**对应的桶

$O(n)$ //模余法

在各桶中，动态记录**最左点**、**最右点**

$O(n)$ //可能相同甚至没有

算出相邻的非空桶之间的“**距离**”

$O(n)$ //一趟遍历足矣

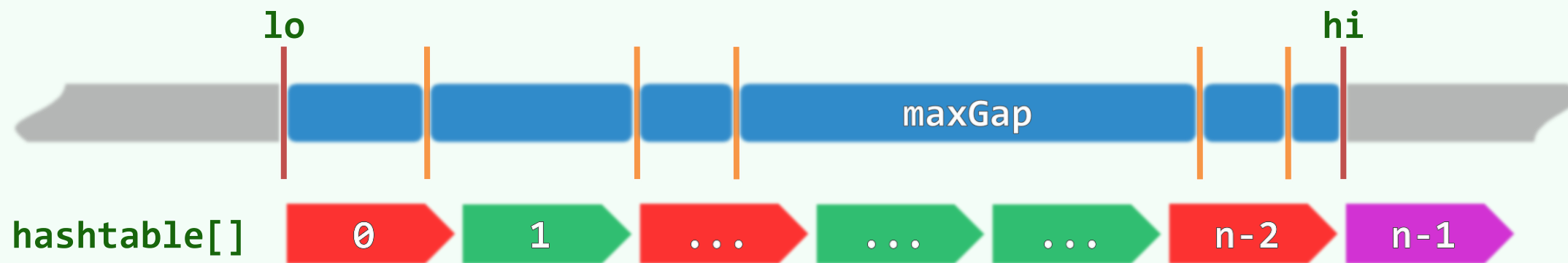
最大的距离即MaxGap

$O(n)$ //画家算法

正确性

❖ 事实：任一数据集中的**最大值**，绝不至于小于**平均值**

于是：MaxGap不可能局限于**某个桶内**，即必然介于**两个非空的桶之间**



❖ 对称的MinGap问题：n-1段有界区间中，何者**最短**？

可否沿用上法，以突破 $\Omega(n \log n)$ 下界？

词典

基数排序 + 整数排序

乃施教法于邦國、都鄙，使之各以教其所治民：令五家為比，使之相保；五比為閭，使之相愛；四閭為族，使之相葬；五族為黨，使之相救；五黨為州，使之相賙；五州為鄉，使之相賓

God made the integers; all else is the work of man.

- L. Kronecker

邓俊辉

deng@tsinghua.edu.cn

多字段 ~ 词典序

❖ 有时，关键码由多个位组成：

$$key = (k_d, \dots, k_3, k_2, k_1)$$

亦称域、字段

比如桥牌：(花色, 点数)

❖ 若将各位视作字母

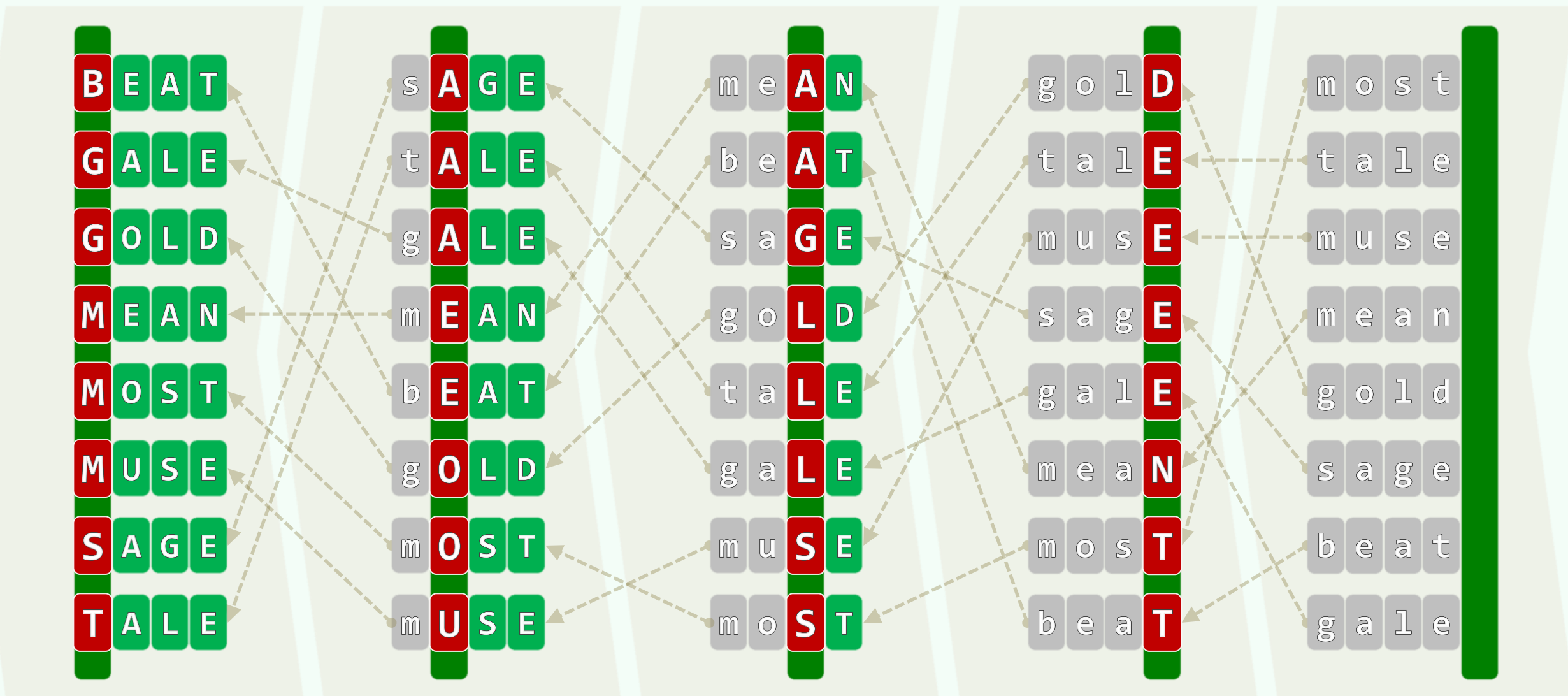
则关键码即单词



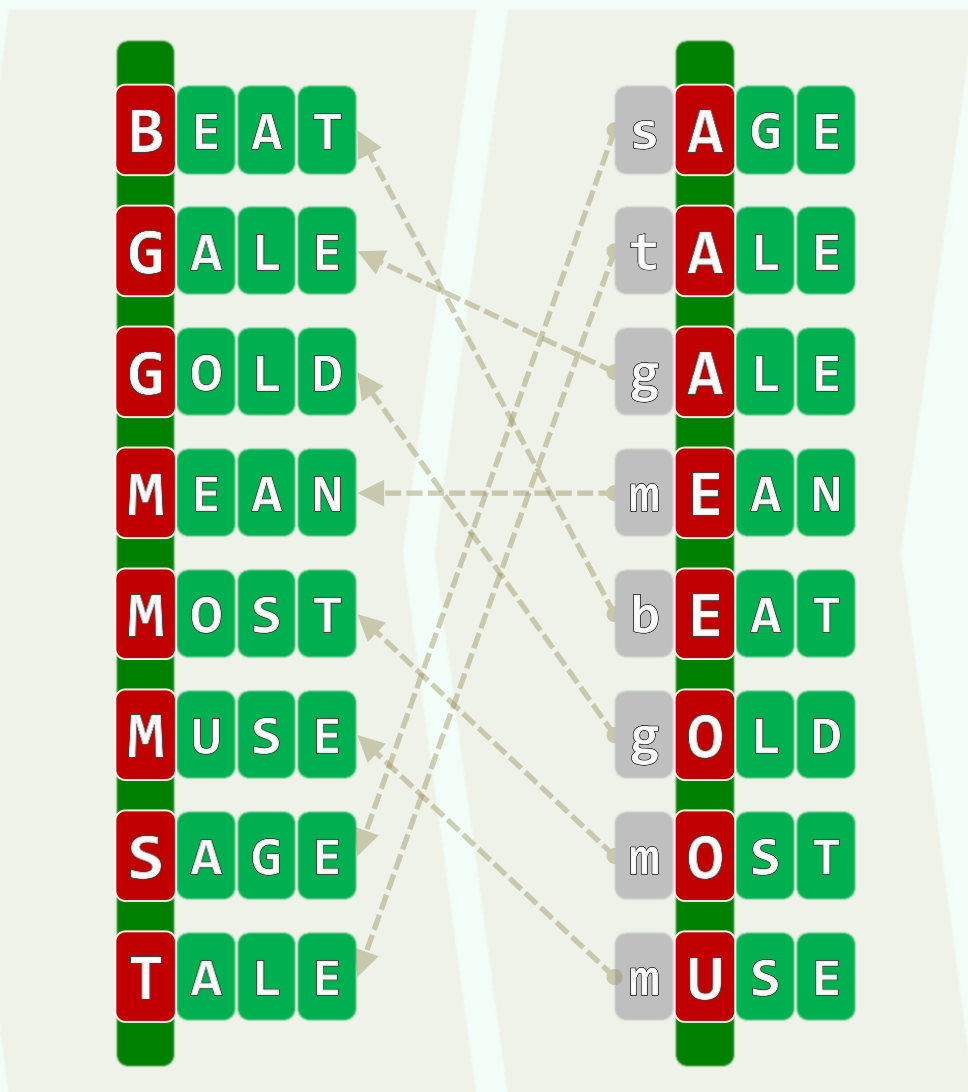
❖ lexicographic order: 按词典的方式排序: algorithm < data < debug < structure

比如，清华大学的校友花名册，排序依据依次为：首入校年 | 院系（编号） | 姓名

算法：自 k_1 到 k_d （**低位优先**），依次以各位为序做一趟**桶排序**



正确性：源自稳定性



❖ 归纳假设：前 i 趟排序后，所有词条关于低 i 位有序

❖ 第1趟显然；现考查第 $i+1$ 趟排序之后的时刻...

❖ 声称：任意一对词条，均满足归纳假设

❖ 实际上，无非两种情况：

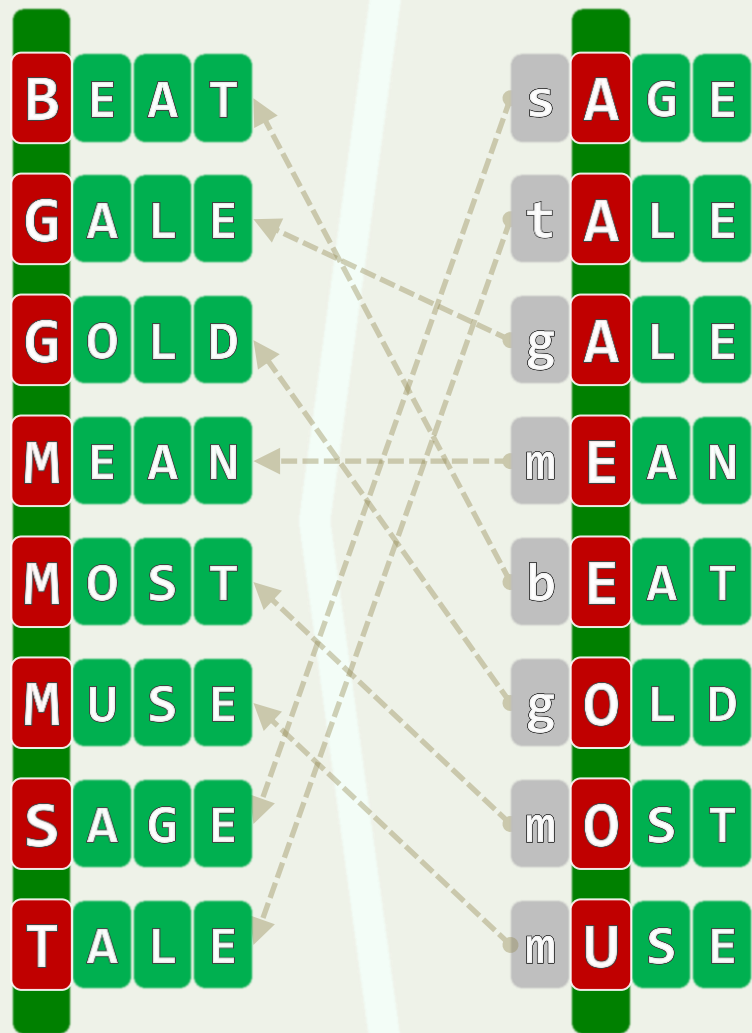
- 二者的第 $i+1$ 位不同 // Muse | Sage

即便此前曾是逆序，现在亦必已转为有序

- 二者的第 $i+1$ 位相同 // gALE | gOLD

得益于桶排序的稳定性，亦必保持原有次序

时间成本



= 各趟桶排序所需时间之和

$$= n + 2m_1$$

$$+ n + 2m_2$$

$$+ \dots$$

$$+ n + 2m_d \quad // m_k \text{ 为各位的取值范围}$$

$$= O(d \times (n + m))$$

$$// m = \max\{m_1, \dots, m_d\}$$

❖ 当 $m = O(n)$ 且 d 可视作常数时, $O(n)$!

❖ 在一些**特定**场合, Radixsort非常高效...

实现（以二进制无符号整数为例）

```
❖ typedef uint32_t U; //约定：类型T或就是U；或可转换为U，并依此定序

❖ template <typename T> void List<T>::radixSort( LNP<T> h, LNP<T> t ) {

    Rank i = 0, n = *t - h - 1; //节点总数

    for ( U radixBit = 0x1; radixBit; radixBit <<= 1 ) //反复地根据当前基数位

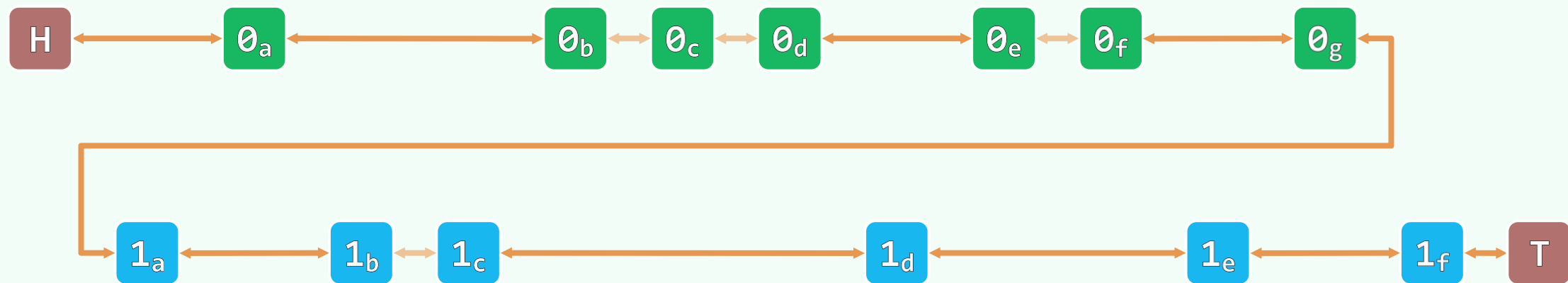
        for ( LNP<T> p = *h + (i = 0); i < n; i++ ) //将所有节点

            radixBit & U (p->succ->data) ? //分拣为前缀（0）与后缀（1）

                p->succ->movePred( t ) : p = p->succ; //将节点p->succ直接转移至t之前

    } //radixSort
```

实例



常对数密度的整数集

❖ 设 $d > 1$ 为常数

❖ 考查取自 $[0, n^d)$ 内的 n 个整数

- 常规密度 $= n/n^d = 1/n^{d-1}$

- 对数密度 $= \frac{\ln n}{\ln n^d} = \frac{1}{d} = \Omega(1)$

❖ 亦即，这类整数集的对数密度至少是**常数**

❖ 这一附加条件，在实际应用中**不难**满足...

❖ 若取 $d = 4$ ，则即便是**64**位整数

也只需 $n > (2^{64})^{1/4} = 2^{16} = \mathbf{65,536}$

❖ 对于这类整数集

有没有效率为 $o(n \log n)$ 的排序算法？

线性排序算法

❖ 预处理:



将所有整数转换为n进制形式: $x = (x_d, \dots, x_3, x_2, x_1)$

❖ 于是, 每个整数均转化为d个位, 故可直接套用Radixsort算法

❖ 排序时间 = $d \cdot (n + n) = \mathcal{O}(n)$ //“突破”了此前确定的下界!

❖ 原因在于:

- 整数取值范围有限制
- 不再是基于比较的计算模式

❖ 进制转换需要多少时间? 回忆一下此前的相关内容...

Θg-F

词典

计数排序

邓俊辉

deng@tsinghua.edu.cn

The purpose of computing is insight, not numbers.

- Hamming

算法 (1)

❖ 回忆：基数排序中反复做的桶排序...

❖ 亦属 “小集合 + 大数据” 类型，是否可以更快？



❖ 仍以纸牌排序为例 ($n \gg m = 4$)

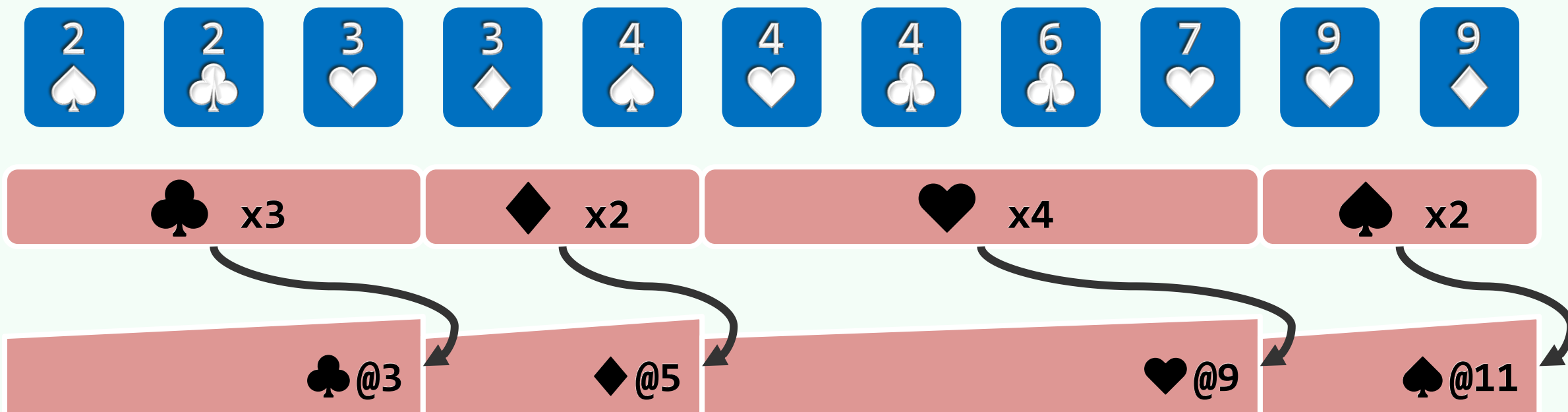
假设已按**点数**排序，以下（**稳定地**）按**花色**排序

1) 经过分桶，**统计**出各种花色的数量 $O(n)$

算法 (2+3)

2) **自前向后**扫描各桶，依次累加 //cumulative sum, $O(m)$

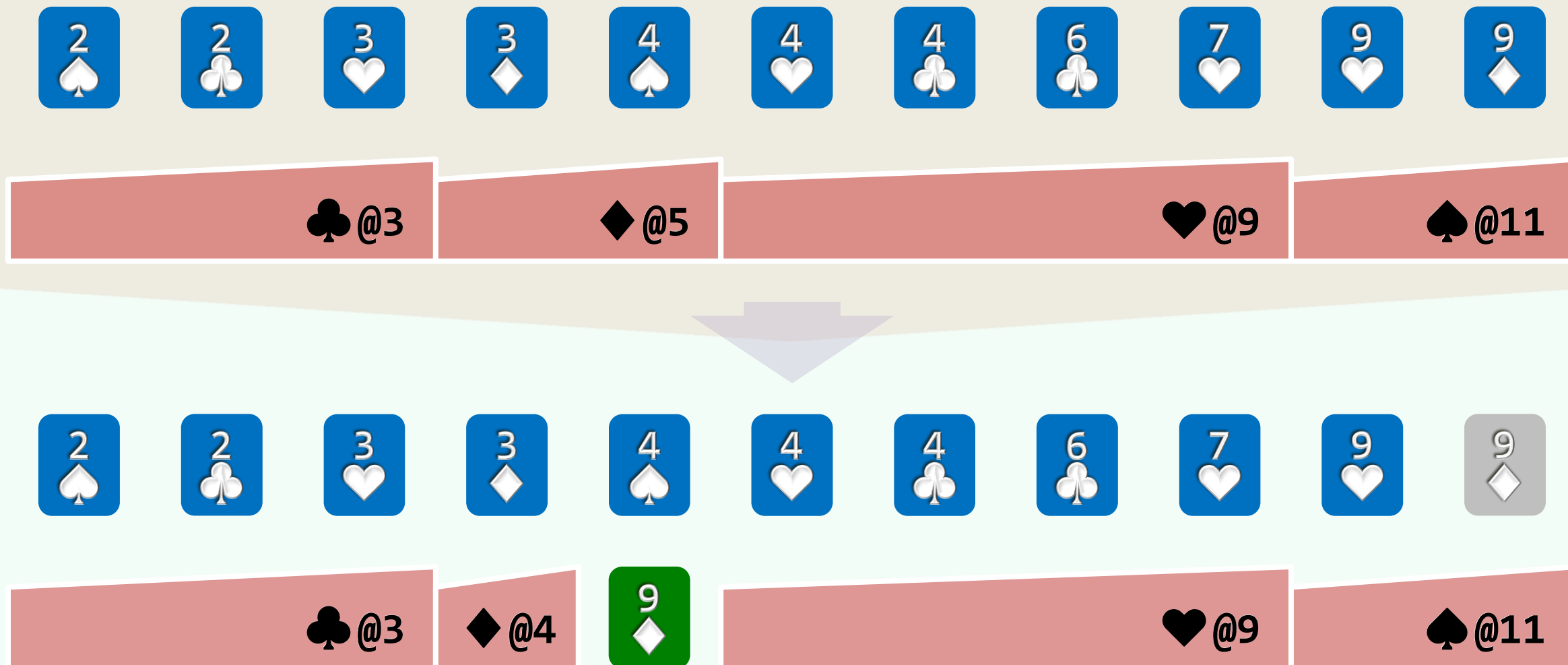
即可确定各套花色所处的**秩区间**: $[0, 3) + [3, 5) + [5, 9) + [9, 11)$



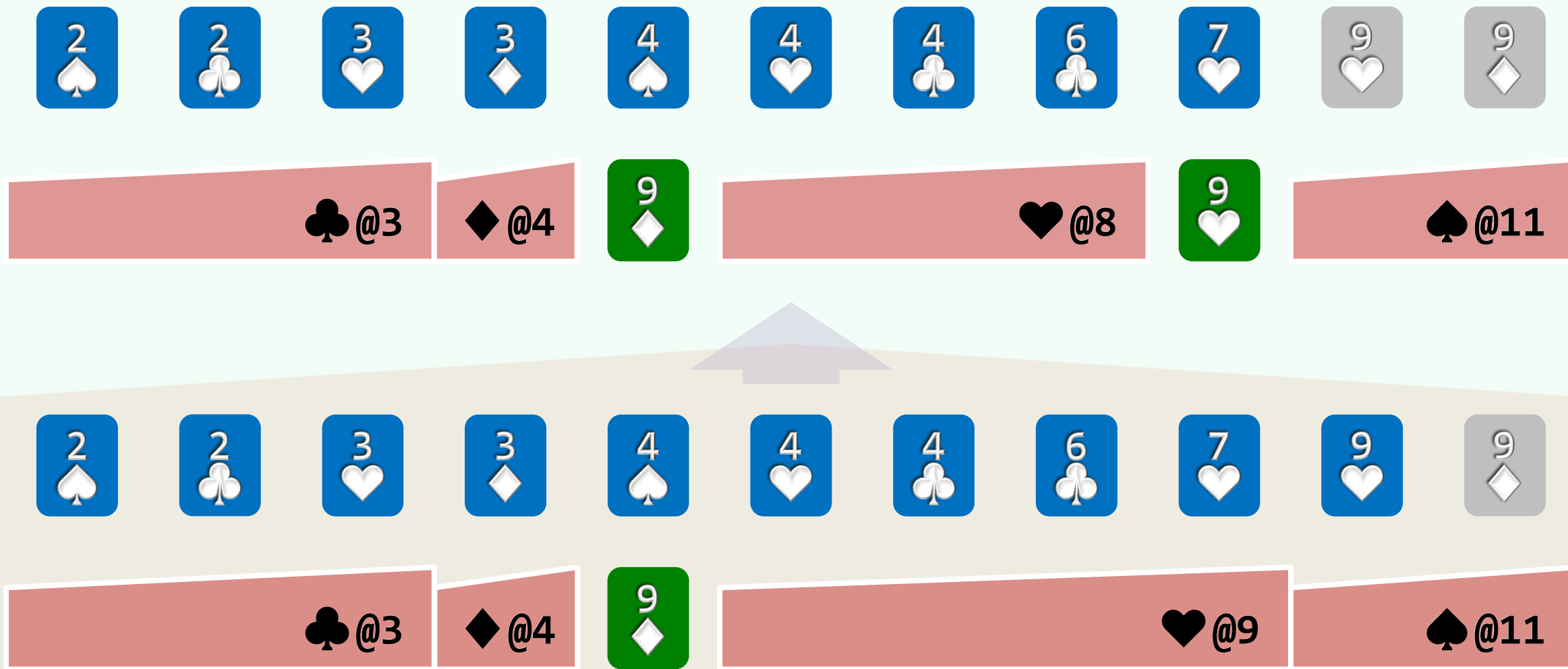
3) **自后向前**扫描每一张牌 // $O(n)$

对应桶的计数**减一**，即是其在最终有序序列中对应的**秩**

实例 (1/11)



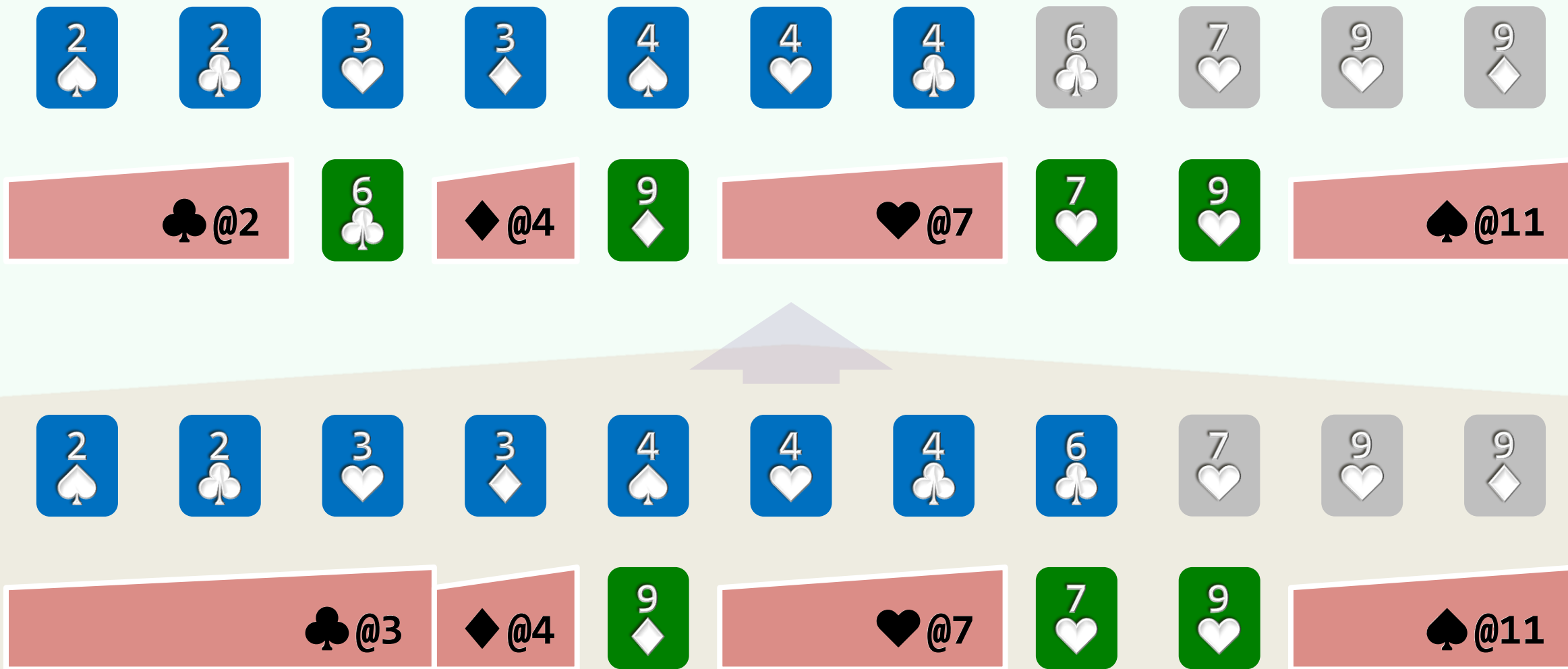
实例 (2/11)



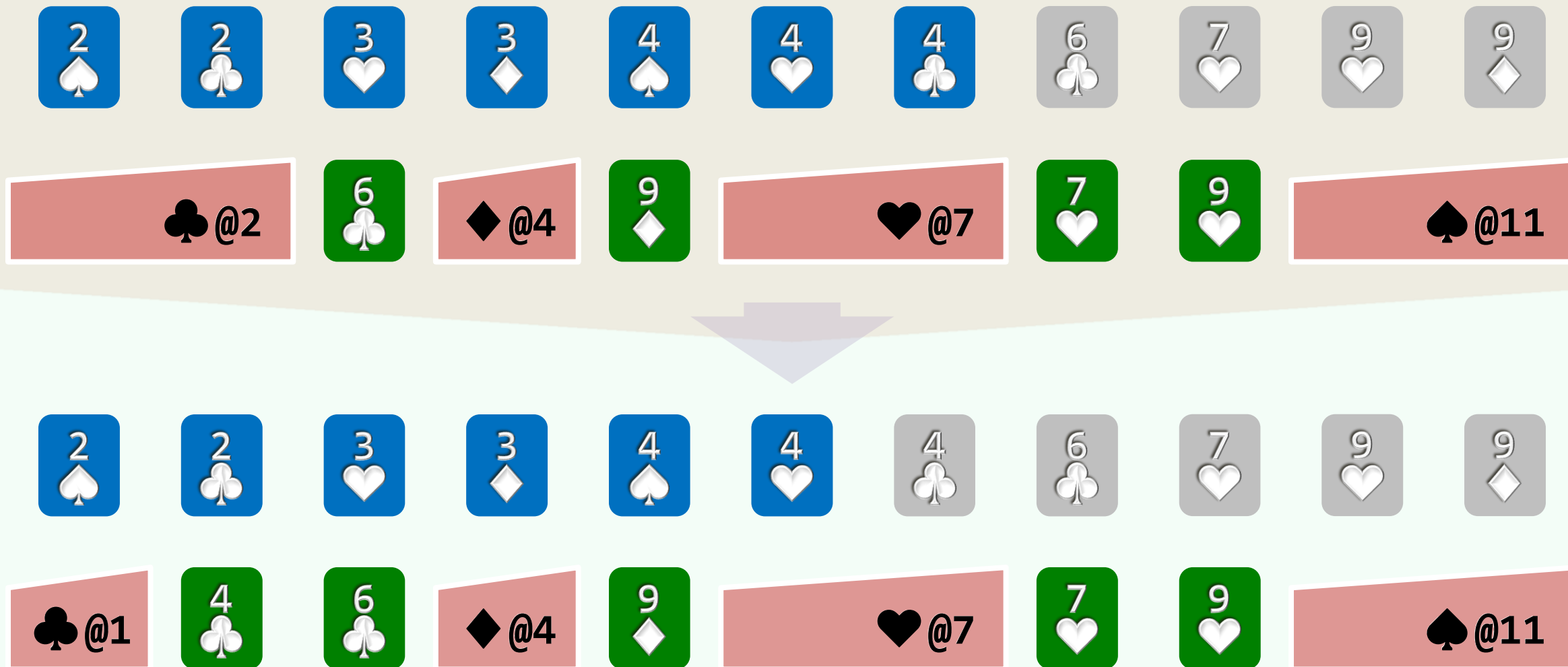
实例 (3/11)



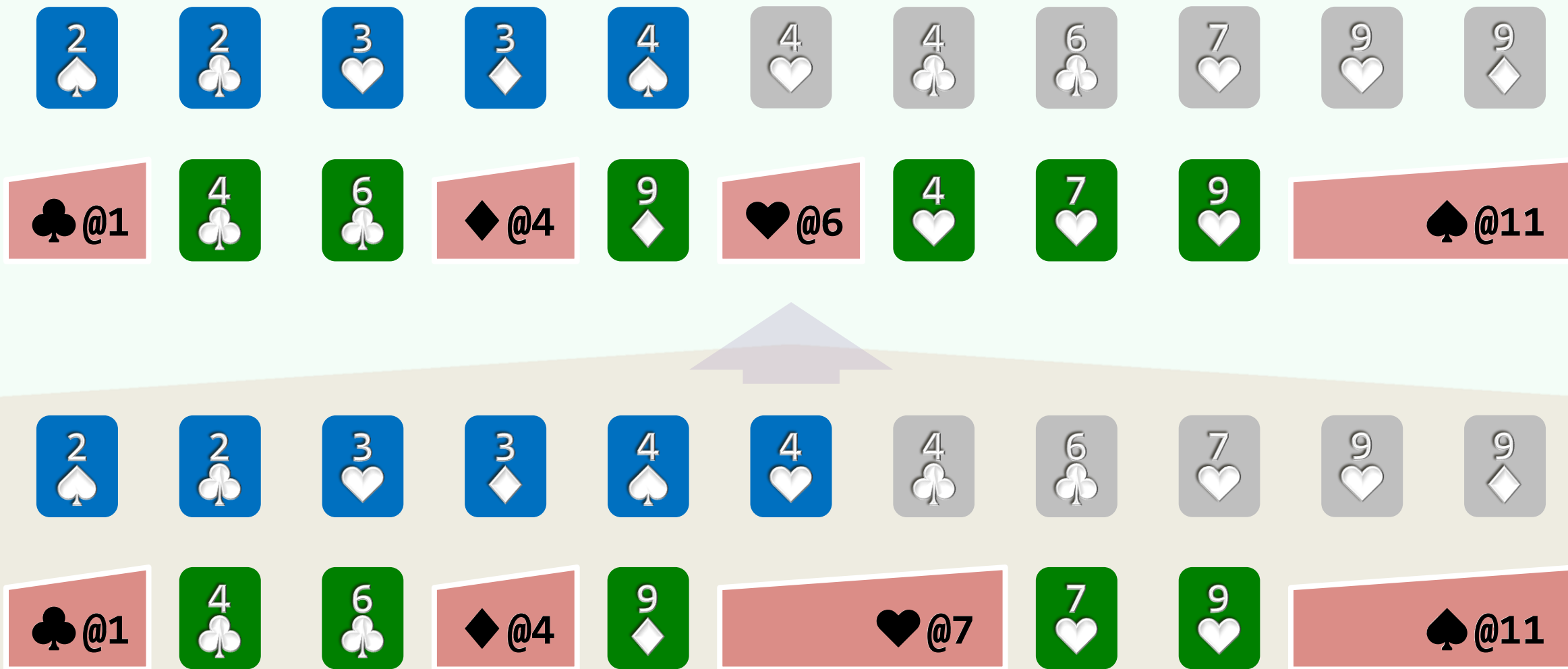
实例 (4/11)



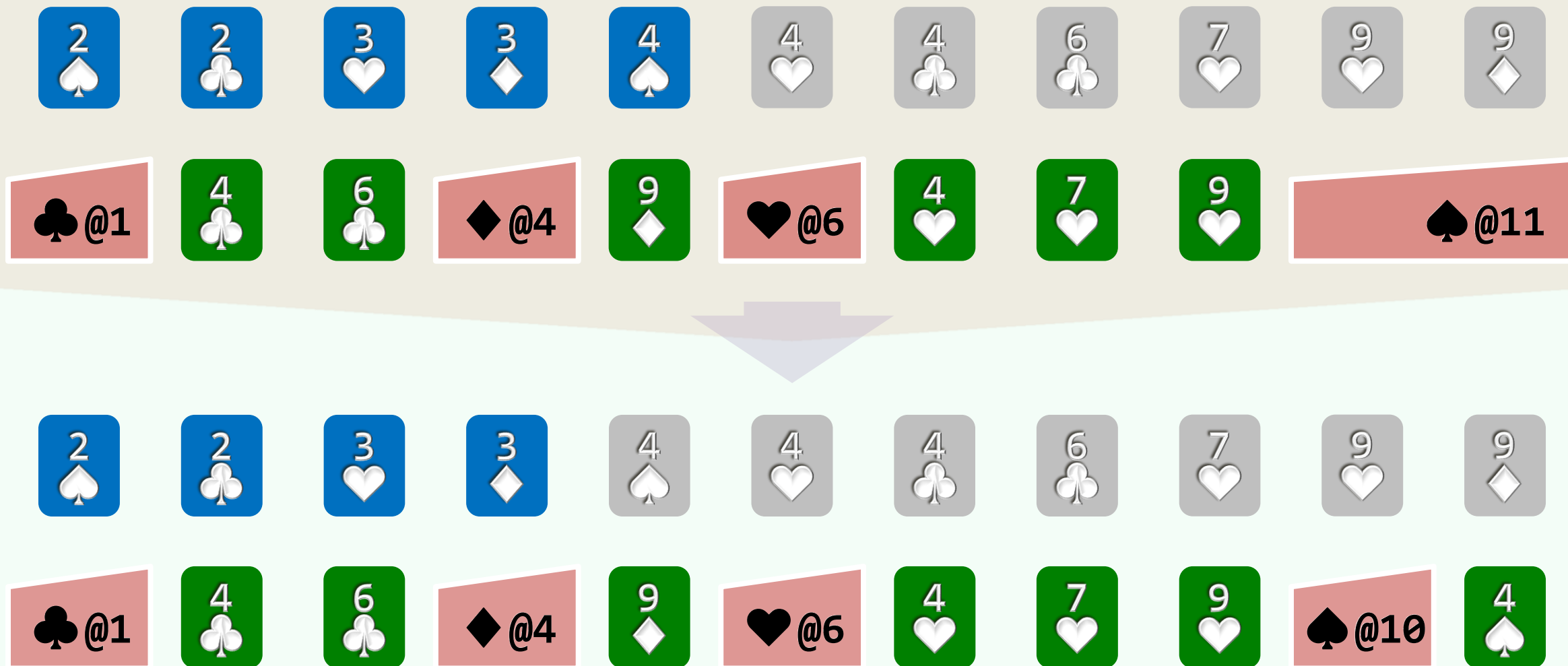
实例 (5/11)



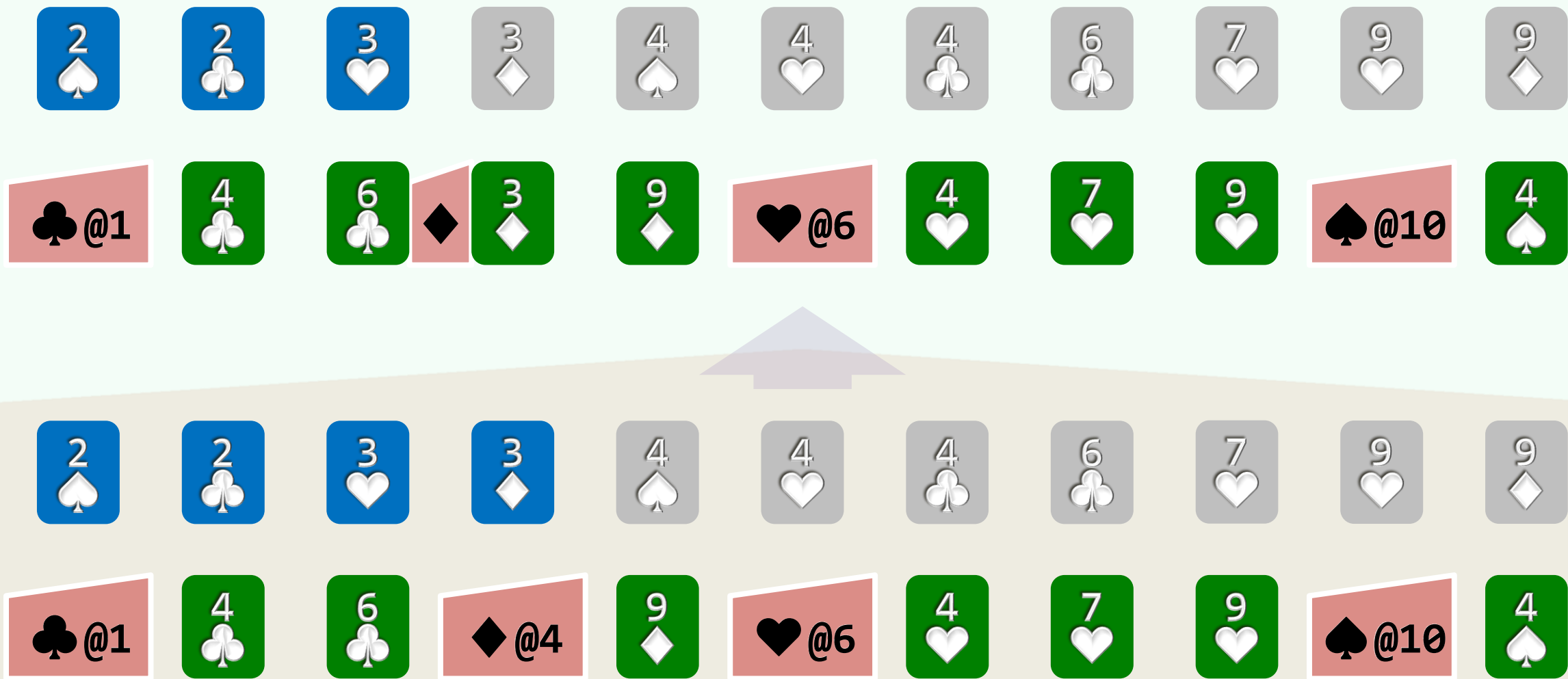
实例 (6/11)



实例 (7/11)



实例 (8/11)



实例 (9/11)



实例 (10/11)



实例 (11/11)



分析

- ❖ 时间复杂度 = $\mathcal{O}(n + m + n) = \mathcal{O}(n)$ ——高效处理大规模数据
- ❖ 空间复杂度 = $\mathcal{O}(n)$ ——不能就地完成，需借助外存
- ❖ 最后一步的扫描次序，可否改为自前向后？

